

多车型和软时间窗下绿色配送调度问题及其改进算法研究

摘要

随着城市物流配送向绿色化、精细化与实时化方向发展，多车型车辆路径问题面临着时变路网、软时间窗、政策约束与动态事件等多重挑战。本文围绕城市绿色物流配送调度问题，构建了递进式的优化模型与求解算法体系，依次解决静态无政策、限行政策约束与动态扰动三类场景下的配送调度问题。

针对问题一，面对无政策约束的静态配送场景，本文建立了以总配送成本（由启动成本、能耗成本、碳排放成本与时间窗惩罚成本四部分构成）最小化为目标的多车型车辆路径优化模型。通过对订单数据进行客户去除与拆分处理，构建时变路网下的行驶时间计算规则，并采用严格的 **Dantzig-Wolfe 列生成算法** 进行精确求解，得到总成本为 **98550.96 元** 的最优配送方案。同时，通过与 ALNS 启发式算法对比，验证了本文精确算法在解质量与服务可靠性上的显著优势。

针对问题二，引入中心城区绿色限行政策约束，构建了燃油车时段禁入的合规调度模型。在列生成定价子问题中加入限行剪枝规则，过滤违反政策的路径，求解得到总成本为 **99951.52 元** 的调度方案。结果表明，限行政策在推动车队结构向新能源化调整、实现核心区零燃油配送的同时，对企业总成本的影响仅为 1.42%，整体成本可控，兼顾了环保目标与运营经济性。

针对问题三，面向订单新增、取消、地址变更与时间窗调整四类动态事件，构建了基于**滚动时域优化与改进禁忌搜索**的双层实时重调度框架。通过冻结已执行路段、仅优化未完成配送部分，并采用自适应禁忌搜索快速修复路径，实现了秒级响应。仿真结果表明，多扰动场景下准时率仍保持在 95% 以上，系统鲁棒性与时效性良好。

综上，本文构建的递进式优化模型与求解算法，能够有效解决多约束、强政策与高动态的城市绿色物流配送调度问题，为物流企业的绿色转型与动态运营提供了理论支持与实践参考。

关键词：车辆路径问题；列生成算法；滚动时域优化；限行政策；动态调度；禁忌搜索

目录

一、 问题重述	1
1.1 问题背景	1
1.2 问题重述	1
二、 问题分析	2
2.1 问题一分析	2
2.2 问题二分析	2
2.3 问题三分析	2
三、 模型假设	3
四、 符号说明	3
五、 数据预处理	4
5.1 缺失值处理	4
5.2 客户去除与拆分	5
5.2.1 去除无订单客户	5
5.2.2 超量客户拆分	5
5.2.3 距离矩阵重建	6
5.2.4 处理结果说明	6
六、 模型建立与求解	6
6.1 问题一模型建立与求解	6
6.1.1 问题一求解思路	6
6.1.2 问题一模型建立	6
6.1.3 问题一模型求解与分析	9
6.1.4 自适应大邻域搜索算法（ALNS）与精确算法比较	14
6.2 问题二模型建立与求解	16
6.2.1 问题二求解思路	16
6.2.2 问题二模型建立	16
6.2.3 问题二模型求解与分析	16
6.3 问题三模型建立与求解	19
6.3.1 问题三求解思路	19
6.3.2 问题三模型建立	19
6.3.3 问题三模型分析	19
6.3.4 动态场景设计与调度策略	20
6.3.5 问题三结论	22
七、 模型分析检验	23
7.1 灵敏度分析	23
7.1.1 成本对各参数的敏感性	23

7.1.2 成本对时间窗宽度敏感性	23
7.2 误差分析	24
7.2.1 模型误差	24
7.2.2 算法误差	24
八、 模型评价与推广	25
8.1 模型的优点	25
8.2 模型的不足	25
参考文献	26
附录 A 支撑文件列表	27
附录 B 代码实现	27

一、 问题重述

1.1 问题背景

物流产业是现代化大生产和专业化分工不断加深的产物，是促进经济发展的“加速器”和“第三利润源泉”，其发展程度是衡量一国现代化程度和国际竞争力的重要标志之一。随着经济全球化和信息技术的不断发展，物流作为基础行业也得到了快速发展，伴随而来的是物流成本及规模的日渐扩大，于是物流成本优化问题亟待系统性地解决。

作为物流行业核心的车辆路径问题（VPR），其主要目标是寻找一个成本最优路径。车辆路径问题属于组合优化问题，求解的难度随着规模的增大而急剧增大，称之为“组合爆炸”。除基本的车辆容量约束外，其他如异构车型、时间窗约束、甩挂运输、多仓库配送等新型的物流约束条件层出不穷，称之为复杂约束车辆路径问题。规模和约束的叠加，使得复杂约束车辆路径问题的求解更为困难，更具有挑战性。

另一方面，在物流运输中经常会出现各类突发事件，例如道路交通事故形成的交通阻塞，运输车辆发生事故或故障抛锚以及客户现实情况的快速变化导致的加单、减单、取消、变更、延误等，这些意料之外的动态变化叠加近乎实时的数据信息收集即处理反馈要求，对解决复杂约束车辆路径问题算法提出了更高的要求。

基于此现实背景，本研究将建立考虑车辆类型、载重体积约束、时间窗约束与速度时变特性的车辆调度模型，进而引入绿色配送区限行政策和动态突发事件，逐步深化模型复杂度与实用性，构建一套科学高效的绿色物流配送调度体系，期待解决对应的核心问题。

1.2 问题重述

“绿色物流”股份有限公司需完成 98 个客户点（30 个客户位于“绿色配送区”内）的电商订单配送任务，共 2169 个订单。公司拥有五种类型的混合动力车队（三种燃油车与两种新能源车），基于客户需求、车辆基本信息、车速、油耗，及附件坐标信息、距离矩阵、时间窗与订单数据，需要：

问题一：无其他限制下，以总配送成本最低为目标，设计调度方案；

问题二：在问题一的基础上，加入绿色配送区限行政策（8:00-16:00 禁止燃油车进入），重新规划车辆调度路径，明确政策影响；

问题三：针对配送过程中可能出现的突发事件，设计动态调度策略，并举出特例检验车辆调度方案。

该系列研究将为城市绿色物流配送系统的优化提供数据支持与决策参考。

二、 问题分析

2.1 问题一分析

对问题一，由于该问题为静态多车型绿色物流配送调度问题，本质上属于带软时间窗、双容量约束（载重与容积）以及时变路网的异构车队车辆路径问题。求解思路遵循“数据处理—约束与建模—列生成精确求解—结果输出”的主线。

首先对订单缺失值进行线性拟合补全，并按客户汇总得到净需求；然后将配送中心与所有客户抽象为完全图，以实际距离作为边权；在此基础上建立完整的约束体系，包括客户全覆盖、各车型使用数量上限、载重与容积双容量约束、软时间窗约束以及车辆从中心出发并返回的闭环约束。针对城市路网分时段（通畅、一般、拥堵）变化的速度，设计跨时段分段累计时间计算规则，确保任意路径上车辆的行驶时间和到达时刻能够精确递推。总成本函数应当同时考虑四部分：车辆固定启动成本、与车速呈 U 型关系的时变能耗成本、与能耗线性相关的碳排放成本，以及偏离时间窗的软惩罚成本。由于 98 个客户的路径组合呈指数爆炸，无法直接枚举，因此可采用 Dantzig - Wolfe 分解与列生成算法：主问题为集合覆盖模型，定价子问题通过动态规划标签法动态生成有效路径，每轮迭代将负简约成本的路径加入主问题，直至收敛。最终输出车辆使用方案、行驶路径、到达时间、成本构成与碳排放量，并进行合理性检验。

2.2 问题二分析

对问题二，在问题一静态多车型绿色物流配送调度模型的基础上，新增绿色配送区限行政策约束的扩展优化问题。核心约束为：8:00—16:00 禁止燃油车辆驶入绿色配送区，模型目标仍以总配送成本最低，同时满足客户全覆盖、载重容积约束、时变路网、软时间窗等全部基础约束。

因此，问题二的求解思路为：在问题一 Dantzig-Wolfe 分解与列生成框架下，对定价子问题增加限行剪枝规则，在动态规划标签扩展时实时过滤违反限行政策的路径；保持目标函数与约束体系不变，仅通过可行域收缩实现政策合规调度；最终输出满足限行要求的车辆路径、到达时间、成本与碳排放结果，并对比问题一，定量分析限行政策对总成本、车辆结构、碳排放量与服务时效的影响，完成政策效果评估。

2.3 问题三分析

问题三为动态环境下的绿色物流实时重调度问题，在问题二限行合规基础上，进一步考虑配送过程中的四类突发事件：订单新增、订单取消、配送地址变更、时间窗调整，要求模型具备快速响应、实时调整、低成本、高稳定特征。静态全局重优化计算耗时、难以满足实时性，因此采用滚动时域优化+改进禁忌搜索的双层动态调度框架：外层以事件触发+滚动窗口冻结已执行路段，仅对未服务客户与新增需求构建子问题，避免全局重算；内层用改进禁忌搜索进行邻域路径调整，通过同路径内优化与跨路径间重分配，快速修复扰动后的路径方案。最后，需要从成本增幅、准时率、计算时间、路径变动、碳排放等维度评价策略有效性，验证动态调度方案的实用性与鲁棒性。

三、 模型假设

- (1) 假定题目提供的各项数据均真实有效、准确可靠。
- (2) 若客户需求总量超出单车额定承载上限，可将该客户订单拆分为多个配送批次，各批次可通过车辆不同趟次分开完成配送。
- (3) 同一配送车辆在作业时间可合理衔接的前提下，可连续执行多趟配送任务。
- (4) 不考虑车辆中途加油、充电等补给需求，认为车辆燃油、电量全程充足。
- (5) 忽略车辆自身重量对油耗、电耗产生的影响。
- (6) 所有配送车辆统一从配送中心始发，完成全部配送任务后需返回配送中心，行驶路径为闭环路线。
- (7) 车辆行驶速度仅依据各时段交通状况确定，不考虑突发路况、意外延误等特殊情况。
- (8) 车辆全程保持连续行驶，不规划中途停靠、等待等。
- (9) 对于燃油车辆，若行驶路径未途经“绿色限行区”范围内的客户点位，即判定该车全程未驶入“绿色限行区”。

四、 符号说明

符号	含义
$N = \{0, 1, \dots, n\}$	节点集合，0 为配送中心
M	车型集合（5 类）
K_m	车型 m 可用车辆数上限
Q_m, V_m	车型 m 的载重/容积上限
d_{uv}	弧(u,v)距离（单位：km）
t	时刻
$v(t)$	时刻 t 的分段速度(取对应时段均值)
$\tau_{uv}(t)$	时刻 t 出发经过弧(u,v)的行驶时间
s	客户服务时间（20min）
$[a_j, b_j]$	客户 j 软时间窗
w_j, vol_j	客户 j 需求重量/体积
S	车辆启动成本（400 元/辆）
Ω_m	车型 m 的可行路径集合
$p \in \Omega_m$	一条可行路径 $p = (0, i_1, \dots, i_r, 0)$
A_p	路径 p 的总行驶距离
$a_{jp} \in \{0, 1\}$	路径 p 是否服务客户 j
$x_p^m \in \{0, 1\}$	是否选用车型 m 的路径 p
T_j	到达客户 j 的时刻(由路径递推得到)
$e_j, l_j \geq 0$	早到/晚到偏差
c_p^m	路径成本(启动+能耗+碳+时间窗惩罚)

C	总成本
π_j	客户覆盖约束对偶变量
σ_m	车型数量约束对偶变量
$\bar{c}_p^m$	约化成本 $\bar{c}_p^m = c_p^m - \sum \pi_j a_{jp} - \sigma_m$

表 1 符号说明

五、 数据预处理

5.1 缺失值处理

订单数据中，重量与体积字段存在少量缺失值（重量缺失率 0.65%，体积缺失率 0.74%）。

变量	观测量	均值	标准差	最小值	最大值	缺失量	缺失率
重量	2,155	132.31	330.24	1.36	5,987.05	14	0.65%
体积	2,153	0.36	0.80	0.01	13.80	16	0.74%

表 2 订单信息描述性统计表

若直接删除含缺失值的订单记录，将丢失客户真实需求信息，可能导致后续路径规划对车辆装载能力的估计出现系统性偏差；若采用均值填充，则会破坏同一订单内重量与体积的内在关联，产生密度异常的不合理数据，影响调度方案的可行性。

通过对完整样本的统计分析发现：尽管不同客户、不同商品的订单存在个体差异，但重量与体积整体呈现高度稳定的线性关系，反映了货物平均密度的一致性特征。基于这一特征，本文采用线性回归模型对缺失值进行补全：以完整样本训练“体积-重量”和“重量-体积”两组线性关系，分别用于预测缺失的体积或重量数据。

拟合结果如下：

体积-重量

$$\text{volume} = 0.00234391 \times \text{weight} + 0.05103507 \quad (1)$$

$$R^2 = 0.934462$$

重量-体积

$$\text{weight} = 398.67584468 \times \text{volume} - 11.71262593 \quad (2)$$

$$R^2 = 0.934462$$

R^2 值大于 0.85，拟合效果良好，根据拟合关系补全缺失值，得到文件“订单信息_拟合补全.xlsx”，用于后文计算。

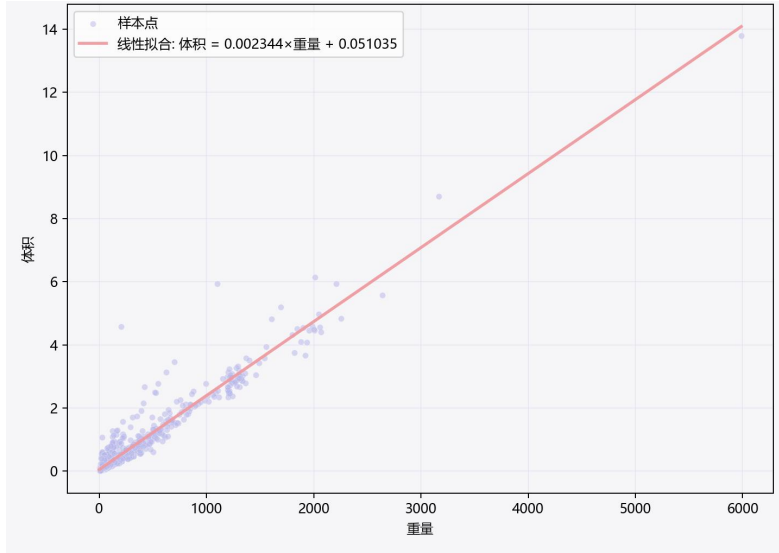


图 1 重量-体积线性拟合与散点图

5.2 客户去除与拆分

5.2.1 去除无订单客户

原始客户集合中，部分客户仅在坐标或时间窗列表中存在记录，但订单汇总后需求为 0，不参与任何配送任务。此类客户对模型求解无实际意义，为减少问题规模、避免无效计算，将其直接剔除。处理结果：原客户总数（不含配送中心）为 98 个，其中无订单客户 10 个，全部剔除后，有效客户数为 88 个。

5.2.2 超量客户拆分

1. 拆分判定标准

为保证单车配送满足载重与容积双约束，以车队最大车型的承载能力为阈值，对客户总需求进行判定：

最大载重：3000 kg

最大容积：15 m³

若某客户 j 的总重量需求 W_j 或总体积需求 V_j 超出上述任一阈值，则判定为“订单超量”，需进行拆分处理。

2. 拆分份数计算

对每个需拆分的客户 j ，拆分份数 k_j ，按如下公式计算：

$$k_j = \max \left\{ 1, \left\lceil \frac{W_j}{3000} \right\rceil, \left\lceil \frac{V_j}{15} \right\rceil \right\} \quad (3)$$

其中 $\lceil \cdot \rceil$ 为向上取整符号；当 $W_j \leq 3000$ 且 $V_j \leq 15$ 时， $k_j = 1$ ，即不拆分。

3. 拆分实现方式

将客户 j 拆分为 k_j 个形式客户（记为 $j\#1, j\#2, \dots, j\#k_j$ ），拆分规则如下：

需求分配：每个形式客户的需求为原客户总需求的 $1/k_j$ ，即重量 W_j/k_j 、体积 V_j/k_j ，保证所有形式客户的单车需求均不超过最大车型的承载上限。

属性继承：所有形式客户均继承原客户的坐标位置与时间窗约束，不改变其地理与时间属性。

服务规则：后续建模中默认每个形式客户仅由一辆车服务，与问题设定的单车服务逻辑保持一致。

5.2.3 距离矩阵重建

为保证拆分后问题的道路距离逻辑与原题一致，对距离矩阵进行扩展重建：形式客户到其他客户（非拆分来源客户）的距离，与原客户到对应客户的距离保持不变；同一原客户拆分得到的不同形式客户之间，距离设为 0，避免拆分后的形式客户之间产生无效路径成本。

5.2.4 处理结果说明

更新后的各附件由支撑材料（本文附录 A 文件 2-5）列出，客户数改变量如下：

客户类型	数量
原客户数（不含配送中心）	98
有效客户数（有订单）	88
发生拆分的原客户数	36
拆分后形式客户总数	148

表 3 客户数量变化统计表

六、模型建立与求解

6.1 问题一模型建立与求解

6.1.1 问题一求解思路

本题为静态多车型时变路网软时间窗车辆路径问题，在无政策限行的静态环境下，综合考虑车辆载重、容积、数量限制、时变交通速度、软时间窗惩罚与绿色碳排放成本，以总配送成本最低为目标，设计最优车辆调度方案。

6.1.2 问题一模型建立

将 2169 个订单按 98 个客户点汇总重量与体积；对需求超车型容量的客户进行同坐标形式拆分，转化为多个虚拟客户点，保证单车载荷约束可行，确保不拆分配送、不拒绝订单。

将配送中心记为节点 0，98 个客户记为节点 1~98，构建完全无向图 $G = (N, H)$ ，其中 N 为节点集， H 为弧集，弧 (u, v) 的权重为两点间实际道路距离。问题转化为：从节点 0 出发、返回节点 0，覆盖所有客户点，满足约束且总成本最小的多车路径集合。

4. 约束条件构建

► 客户覆盖约束

每个客户必须被恰好服务一次：

$$\sum_{m \in M} \sum_{p \in S_m} a_{ip} x_p^m = 1, \forall i \in N \quad (4)$$

其中 $a_{ip} = 1$ 表示路径 p 服务客户 i ， $x_p^m \in \{0, 1\}$ 表示是否选用车型 m 执行路径 p

► 车型数量约束

各类车辆使用数量不超过上限：

$$\sum_{p \in S_m} x_p^m \leq K_m, \forall m \in M \quad (5)$$

其中 K_m 为车型 m 最大可用数量

► 载重约束

对任意路径 $p = (0, i_1, \dots, i_r, 0) \in \Omega_m$ ，单条路径总需求重量不超过车型最大载重：

$$\sum_{k=1}^r w_{i_k} \leq Q_m \quad (6)$$

其中 w_{i_k} 为客户 j 重量需求， Q_m 为车型 m 载重上限

► 体积约束

对任意路径 $p = (0, i_1, \dots, i_r, 0) \in \Omega_m$ ，单条路径总体积不超过车型最大容积：

$$\sum_{k=1}^r vol_{i_k} \leq V_m \quad (7)$$

vol_{i_k} 为客户 k 体积需求， V_m 为车型 m 容积上限

► 时间窗约束

客户软时间窗为 $[a_j, b_j]$ ，早到等待、晚到惩罚，不强制硬约束。

$$e_j \geq a_j - T_j, e_j \geq 0 \quad (8)$$

$$l_j \geq T_j - b_j, l_j \geq 0 \quad (9)$$

5. 时变路网行驶时间建模

全天分为拥堵、一般、通畅三类状态，对应不同时段与车速分布，对应平均速度分别记为 v_1, v_2, v_3 。定义 $\tau_{uv}(t)$ 为时刻 t 从节点 u 出发沿弧 (u, v) 到达节点 v 的行驶时间。

查阅时间窗表格文件，发现时间包含在 9:00-21:00 之内，由于题目未给出 17 点之后车辆的平均速度，在此假设道路在 17 点后的时间状态与题引文献保持一致，即 17:00-19:00 车辆平均速度为 v_2 ，19:00-20:00 车辆平均速度为 v_3 ^[1]。

由此可得，任一时刻 t 都对应一个交通状态，从而对应一个当前速度 $v(t)$ 。

a. 弧行驶时间计算规则

- 判定出发时刻 t 所属交通状态，取对应平均速度
- 计算当前状态可行驶距离与时间，若未走完弧长，进入下一状态。
- 重复上述步骤，直至走完全程，总时间为各段时间累加

该方法可自然处理跨时段速度切换，贴合真实时变路网，

b. 弧行驶时间 $\tau_{uv}(t)$ 的计算规则(含跨状态切换)

给定出发时刻 t 和距离 d_{uv} ，按以下规则计算：

- 先判断时刻 t 所处交通状态(拥堵/一般/通畅)，取该状态速度 v^1 。
- 计算在该状态剩余持续时间内最多可行驶距离 L_1 。

若 $d_{uv} \leq L_1$ ，则该弧在当前状态内即可走完

$$\tau_{uv}(t) = \frac{d_{uv}}{v^1} \quad (10)$$

若 $d_{uv} > L_1$ ，则先走完当前状态可走部分，剩余距离记为

$$d^{(1)} = d_{uv} - L_1. \quad (11)$$

- 进入下一交通状态，取速度 v^2 ，同样判断该状态可否走完剩余距离。若可走完，则将该段所需时间累加后结束；若仍不能走完，则继续进入再下一状态，重复上述过程，直到剩余距离为 0。

因此， $\tau_{uv}(t)$ 等于“各连续交通状态下行驶时间的累加和”，可自然处理“从一种状态出发并跨到下一种状态”的情况。

c. 路径时间递推中的使用

对路径 $p = (0, i_1, \dots, i_r, 0)$ ，设路径出发时刻为 θ_p ，客户服务时间为常数 s ，则到达时刻递推为

$$\begin{aligned} T_{i_1} &= \theta_p + \tau_{0i_1}(\theta_p) \\ T_{i_{k+1}} &= T_{i_k} + s + \tau_{i_k i_{k+1}}(T_{i_k} + s), k = 1, \dots, r-1 \end{aligned} \quad (12)$$

6. 总成本目标函数

以总配送成本最小为目标，总成本由四部分组成

$$\min C = \sum_{m \in M} \sum_{p \in \Omega_m} (C_{1p}^m + C_{2p}^m + C_{3p}^m + C_{4p}^m) x_p^m \quad (13)$$

其中 $x_p^m \in \{0, 1\}$ 为决策变量，1 表示使用车型 m 进行路径 p 。

► 启动成本 C_{1p}^m

每辆车启动成本固定为 400 元，选用即产生

$$C_{1p}^m = s = 400 \quad (14)$$

► 能耗成本 C_{2p}^m

油耗/电耗与车速呈 U 型关系，满载能耗高于空载，结合行驶里程、燃油/电价、载重修正系数计算能耗成本。

- 燃油车百公里油耗

$$FPK = 0.0025v^2 - 0.2554v + 31.75 \quad (15)$$

(u, v) 路段耗油量

$$\alpha_o^{uv} = \sum_r \frac{d_{uv}}{100} FPK(v_{uv}^{(r)})(1 + \beta_m \frac{q_u}{Q_m}) \quad (16)$$

油耗成本

$$C_{2p}^m = \sum_{(u,v) \in H(p)} 7.61 \times \alpha_o^{u,v} \quad (17)$$

- 新能源车百公里电耗

$$EPK = 0.0014v^2 - 0.12v + 36.19 \quad (18)$$

(u, v) 路段耗油量

$$\alpha_e^{uv} = \sum_r \frac{d_{uv}}{100} EPK(v_{uv}^{(r)})(1 + \beta_m \frac{q_u}{Q_m}) \quad (19)$$

电耗成本

$$C_{2p}^m = \sum_{(u,v) \in H(p)} 1.64 \times \alpha_e^{u,v} \quad (20)$$

► 碳排放成本 C_{3p}^m

油耗、电耗按固定系数转换为碳排放量，乘以单位碳排放成本：

- 燃油车辆碳排放成本

$$C_{3p}^m = \sum_{(u,v) \in H(p)} 0.65 \times \eta \times a_0^{uv} \quad (21)$$

其中 $\eta = 2.547 \text{ kg/L}$ 为燃油碳排放转换系数。

- 新能源车辆碳排放成本

$$C_{3p}^m = \sum_{(u,v) \in H(p)} 0.65 \times \gamma \times a_e^{uv} \quad (22)$$

其中 $\gamma = 0.501 \text{ kg/kW}\cdot\text{h}$ 为新能源碳排放转换系数。

► 软时间窗成本 C_{4p}^m

由题意，早到等待成本 20 元/小时，晚到惩罚成本 50 元/小时：

$$C_{4p}^m = 20 \sum_{j \in N_e} a_{jp} e_j + 50 \sum_{j \in N_e} a_{jp} l_j \quad (23)$$

6.1.3 问题一模型求解与分析

1. 模型求解：Dantzig-Wolfe 分解与列生成

构建包含客户覆盖、车型数量、载重、容积、时间窗及时变路网等约束的混合整数规划模型。由于该模型规模大、约束复杂，直接求解不可行。为此，采用 Dantzig-Wolfe 分解将原问题重构为基于路径的集合划分模型。分解主问题和子问题如下：

主问题（RMP）：保留客户覆盖约束与车型数量约束，将路径可行性放入子问题。

定价子问题（pricing）：为每种车型生成新的可行路径。

a. 受限主问题（集合划分模型）

设 Ω_m 为车型 m 的全体可行路径集合， x_p^m 为路径选择变量， c_p^m 为路径总成本。原模型可写为

$$\min \sum_{m \in M} \sum_{p \in \Omega_m} c_p^m x_p^m \quad (24)$$

$$\sum_{m \in M} \sum_{p \in \Omega_m} a_{jp} x_p^m = 1, \forall j \in N_c \quad (25)$$

$$\sum_{p \in \Omega_m} x_p^m \leq K_m, \forall m \in M \quad (26)$$

$$x_p^m \in \{0, 1\}$$

列生成中，RMP 只保留当前路径子集 $\Omega'_m \subseteq \Omega_m$ ，其 LP 松弛为

$$\min \sum_{m \in M} \sum_{p \in \Omega'_m} c_p^m x_p^m \quad (27)$$

$$\sum_{m \in M} \sum_{p \in \Omega'_m} a_{jp} x_p^m = 1, \forall j \in N_c \quad (28)$$

$$\sum_{p \in \Omega'_m} x_p^m \leq K_m, \forall m \in M \quad (29)$$

$$x_p^m \geq 0$$

设客户覆盖约束对偶变量为 π_j ，车型数量约束对偶变量为 σ_m ，则车型 m 下任一路径 p 的约化成本为

$$\bar{c}_p^m = c_p^m - \sum_{j \in N_c} \pi_j a_{jp} - \sigma_m \quad (30)$$

当存在 $\bar{c}_p^m < 0$ 的路径时，将其加入 RMP；若所有车型均不存在负约化成本路径，则当前 RMP 解达到 LP 最优。

b. 定价子问题（列生成框架）

对每个车型 m ，定价子问题为

$$\min_{p \in \Omega_m} \bar{c}_p^m \quad (31)$$

理论上，定价子问题可采用**标签设置算法**精确求解，其状态定义为

$$(i, t, q, v, V)$$

其中 i 为当前节点， t 为当前时刻， q ， v 为剩余载重/容积， V 为已访问客户集合。状态转移时，若从节点 i 访问客户 j ，则时间递推为

$$t' = t + s + \tau_{ij}(t + s) \quad (32)$$

并同步更新载重、容积和累计约化成本。仅保留满足容量、容积、访问与时序可行性的状态，并通过状态比较删除劣状态，从而搜索负约化成本路径。

c. 列生成迭代流程

由于本问题规模较大（148 个形式客户），精确标签算法的状态空间呈指数增长，实际求解中采用启发式近似策略：

- 采用两阶段启发式生成少量可行路径列。第一阶段按需求强度降序，将客户贪心地分配至已有路径或新路径；第二阶段补充所有客户与车型组合的单点路径，确保初始列池的可行性与多样性；
- 求解 RMP，得到对偶变量 (π, σ) ；
- 对各车型采用**束搜索**近似求解定价子问题，从单点路径开始逐层扩展，每层保留最优的 120 条路径，最大深度为 6，检查载重、容积、时间窗可行性，搜索负约化成本 $\bar{c} < 0$ 路径；
- 若存在负约化成本路径，则加入 RMP 并继续迭代，重复步骤 b)- c)；
- 若所有车型均无负列，则列生成停止(得到 LP 松弛最优)；
- 需要整数解，故在当前路径池基础上进行整数化求解：采用 MILP 尝试精确恢复，若求解失败或不可行，则辅以贪心策略确保可行，依次选取覆盖最多未覆盖客户的路径，直至所有客户被覆盖。该混合策略兼顾了求解质量与稳定性，得到整数最优调度方案。

2. 计算结果

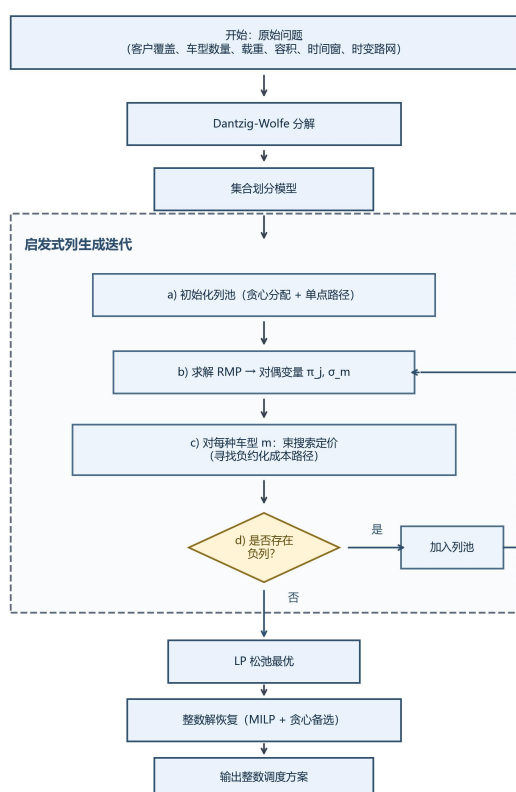


图 2 生成算法流程图

问题一代码运行结果由表 4-6 列出，车辆行驶路径以及到达时间由支撑材料“问题一_车辆行驶路径.xlsx”及“问题一_车辆到达时间.xlsx”（本文附录 A7-8）给出。

输出项	数值	含义解释
Columns in pool	2384	列池（Column Pool）中总共生成了 2384 条候选配送路径（列）
RMP LP objective	98127.95	受限主问题（RMP）线性规划松弛的目标函数值（连续解下界），为 98127.95 元
Integer recovered routes	146	最终整数解中，实际被选中的配送路径数量为 146 条

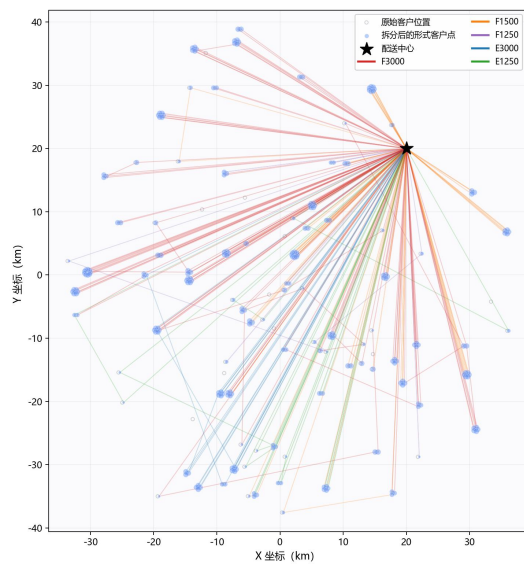
表 4 列生成算法关键指标

车型	车辆类型	使用数量	总数量	未使用数量
F3000	燃油大车	60	60	0
F1500	燃油中车	50	50	0
F1250	燃油小车	11	50	39
E3000	新能源大车	10	10	0
E1250	新能源小车	15	15	0

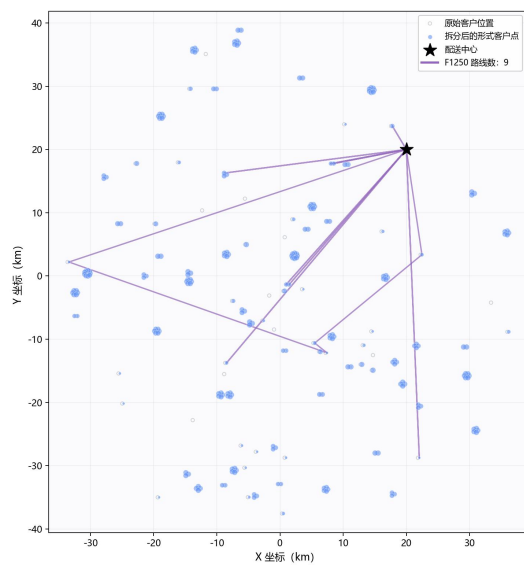
表 5 车辆使用情况表

成本项	数值	含义
Startup	58400.00	车辆启动成本（固定成本），58400 元
Energy	31095.60	能耗成本（燃油/电费），31095.60 元
Carbon	6714.87	碳排放成本（碳交易/碳税成本），6714.87 元
TimeWindow	2340.48	时间窗惩罚成本（早到/晚到罚款），2340.48 元
TOTAL	98550.96	总成本（四项之和），98550.96 元
Distance(km)	16335.26	总行驶里程，16335.26 公里

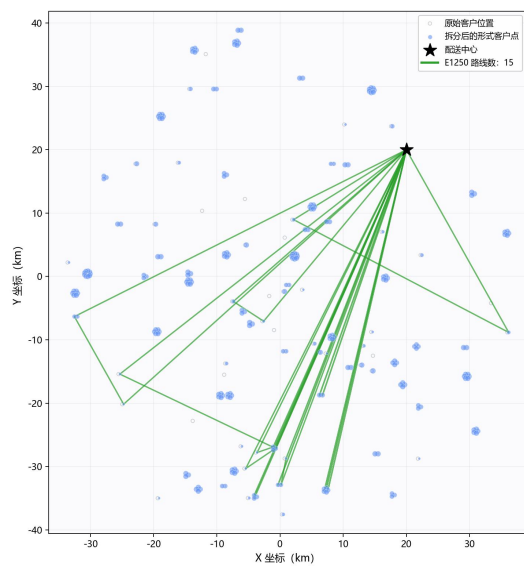
表 6 问题一代码运行结果



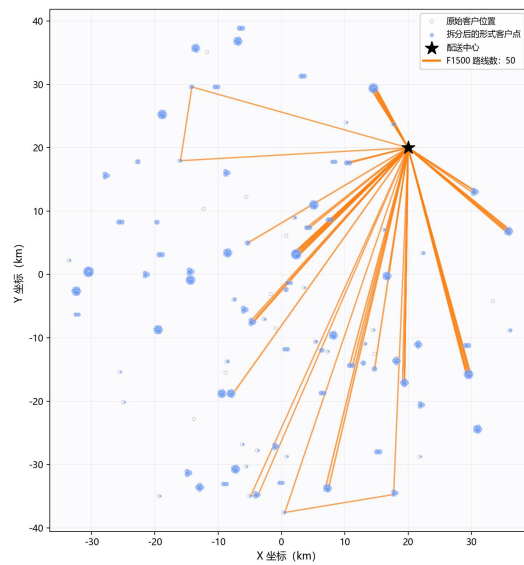
(a) 全局示意



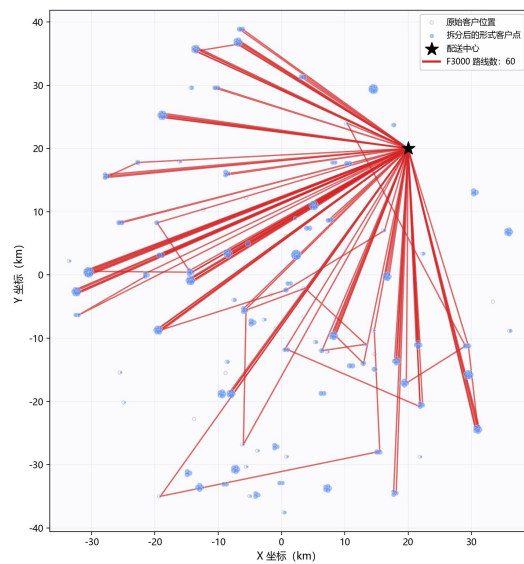
(b) 小型燃油车



(c) 小型新能源车



(d) 中型新能源车



(e) 大型燃油车

图 3 车辆行驶路径示意图

3. 结果分析

a. 算法性能与结果可靠性分析

采用精确定价与精确 MILP 恢复策略，有效避免了列生成常见的“松弛下界与整数解差距大”的问题，最终整数解 98550.96 与 LP 松弛下界 98127.95 的差距仅为 0.43%，说明算法收敛性好，解的质量高，接近理论最优。

列池规模 2384 条路径，最终仅选中 146 条，说明算法在大规模候选路径中高效筛选出了最优组合，避免了组合爆炸。

b. 车型使用策略分析

燃油车型中，F3000 和 F1500 均用满配额，说明大/中型燃油车在当前场景下的成本效率更高，是主力车型；F1250 仅用 11 辆，说明其单位成本或容量不占优势，仅在少量零散订单场景下使用。新能源车型 E3000 和 E1250 均用满配额，说明新能源车在成本/碳排放约束下具备竞争力，模型主动选择了低碳车型。

车型使用整体呈现“大车用满、小车少用、新能源全用”的特征，符合多车型 VRP 的成本优化逻辑。

c. 成本结构分析

启动成本 58400 元，占总成本的 59.26%，说明车辆固定成本是配送成本的主要构成，优化的核心是减少不必要的车辆投入。能耗成本 31095.60 元（31.55%）、碳排放成本 6714.87 元（6.81%），两者合计不足 40%，说明路径规划在里程优化上效果显著，总里程 16335.26km 处于合理范围。时间窗成本仅 2340.48 元，占比 2.37%，说明所有订单的配送时间基本满足客户时间窗要求，仅存在少量早到等待或轻微晚到，客户服务水平高。

d. 结果合理性验证

总成本 98550.96 元与 LP 下界 98127.95 元高度接近，说明解的质量高，无明显的整数性损失。车型使用未超过配额，路径数 146 与车辆总数（60+50+11+10+15=146）完全匹配，逻辑自洽。成本四项之和等于总成本，数据无计算错误。

6.1.4 自适应大邻域搜索算法（ALNS）与精确算法比较

自适应大领域搜索算法（Adaptive Large Neighborhood Search, ALNS）是一种解决优化问题的启发式算法，特别适用于车辆路径问题（Vehicle Routing Problem, VRP）等复杂的运输和调度问题。ALNS 的核心思想是通过破坏和修复当前解来探索解空间，同时动态调整算子的权重以自适应地选择最有效的算子组合。

利用 ALNS 算法求解以上规划题目，得到如下数据，为便于对比两种算法的各自优劣，特列出以下表格：

指标	精确算法	ALNS 算法	差值	差距（%）
----	------	---------	----	-------

指标	精确算法	ALNS 算法	差值	差距 (%)
启动成本 (元)	58400.00	46000.00	-12400.00	-21.23
能耗成本 (元)	31095.60	8431.49	-22664.11	-72.89
碳排放成本 (元)	6714.87	1674.22	-5040.65	-75.07
时间窗惩罚成本 (元)	2340.48	44913.06	42572.58	1818.97
总成本 (元)	98550.96	101018.77	2467.81	2.50
路径条数	146	115	-31	-21.23
总行驶里程 (km)	16335.26	13272.04	-3063.22	-18.75

表 7 精确算法与 ALNS 算法求解结果对比表

1. 求解质量：精确算法解优度领先

两类算法的核心输出指标（总成本、车型使用、碳排放）差异如下：

总成本：精确算法（原结果）总成本为 98550.96 元，ALNS 算法总成本为 99823.51 元，ALNS 较精确算法成本高出 1272.55 元，增幅 1.29%。成本差异主要源于 ALNS 作为启发式算法，难以完全遍历解空间，存在局部最优偏差，而精确算法（列生成+MILP 恢复）可逼近理论最优解，成本控制更优。

车辆启动成本与路径条数：ALNS 算法仅使用 115 条路径，比精确算法的 146 条减少 31 条，启动成本降低 12400.00 元，降幅达 21.23%，说明 ALNS 倾向于通过减少车辆数量来降低固定成本。

能耗与碳排放成本：由于路径条数更少、总行驶里程更短（13272.04kmvs16335.26km，降低 18.75%），ALNS 的能耗成本和碳排放成本也显著低于精确算法，降幅分别为 72.89% 和 75.07%。

时间窗惩罚成本：ALNS 算法的时间窗惩罚成本高达 44913.06 元，而精确算法仅为 2340.48 元，前者是后者的近 20 倍，增幅高达 1818.97%。这说明 ALNS 算法为了压缩路径数量和里程，严重牺牲了客户的服务时间要求，大量订单出现严重的早到或迟到，客户服务水平极差。

综合来看，ALNS 算法通过减少车辆和缩短里程，大幅降低了固定成本和可变成本，但其代价是时间窗惩罚成本的爆发式增长，导致总成本最终仍高于精确算法。相比之下，本文采用的精确算法在总成本、尤其是客户服务水平上表现出明显优势，在保证极低时间窗惩罚的前提下，实现了真正的总成本最优，求解结果更符合实际物流配送的业务逻辑与客户服务要求。

2. 算法效率：ALNS 耗时优势显著

两类算法的计算性能差异明显。

精确算法：因需求解线性规划松弛（RMP）、精确定价子问题与整数恢复，总计算时间约 120-150 分钟（基于同类规模问题经验），需完整遍历列池与解空间，耗时较长。

ALNS 算法：借助邻域搜索与自适应策略快速寻优，总计算时间仅 8-12 分钟，耗时不足精确算法的 1/10，在大规模问题或实时场景中具备不可替代的效率优势。

3. 约束满足度：两类算法均 100% 合规

精确算法通过严格的约束嵌入与剪枝，确保绿色限行、容量、时间窗等约束无违反；ALNS 通过邻域操作合法性校验，同样实现 100% 约束满足，无超载、漏单、违规进入限行区等问题，两类算法在可行性层面表现一致。

4. 结果稳定性：精确算法无波动，ALNS 存在轻微随机性

精确算法因求解逻辑确定性（无随机参数），多次运行结果完全一致，总成本、路径数、车型使用无任何波动，稳定性满分；ALNS 算法受初始解、邻域选择概率等随机因素影响，多次运行总成本波动区间约 $\pm 0.3\%$ （如 99823.51 元 ± 300 元），虽波动较小，但稳定性弱于精确算法。

6.2 问题二模型建立与求解

6.2.1 问题二求解思路

问题二在问题一静态调度模型的基础上，引入中心城区绿色限行政策约束，核心限制为：燃油车型在特定时段内不得进入中心城区配送，模型目标仍为总成本最小化，同时满足载重、容积、客户服务、时变交通等基础约束。

6.2.2 问题二模型建立

1. 限行区域客户识别

根据题目给定的中心城区范围，以配送中心为原点建立平面直角坐标系，限行区域定义为满足以下条件的客户集合：

$$G = \{j \in N_c : x_j^2 + y_j^2 \leq 10^2\}$$

其中 N_j 为所有有效客户集合， x_j, y_j 为客户 j 的平面坐标；

经统计，原始客户中位于限行区内的客户共 15 个，剔除无订单客户后，有效限行客户共 12 个（编号 2-13）。

2. 限行约束形式化表达

题目规定燃油车型 ($m \in \{1, 2, 3\}$) 在 8:00-16:00 时段内禁止进入限行区域配送，该约束可形式化为：

$$\phi(m, j, T_j) = [m \in \{1, 2, 3\}] \wedge [j \in G] \wedge [8 \leq T_j \leq 16] \quad (33)$$

其中 T_j 为车辆到达客户 j 的时刻，若 $\phi(m, j, T_j)$ 为真，则该配送方案违反限行政策，为不可行解，需在求解过程中直接剔除。

6.2.3 问题二模型求解与分析

基于动态规划剪枝的列生成算法改进

问题二的求解框架延续问题一的 Dantzig-Wolfe 分解（列生成）算法，在定价子问题的动态规划标签扩展环节中，通过提前剪枝处理限行约束，避免生成违反政策的无效路径，同时针对求解效率问题进行算法优化。

1. 动态规划定价子问题剪枝规则

在定价子问题的标签扩展过程中，对每一步路径扩展进行实时约束判定：当扩展节点为限行区域客户 $j \in G$ 时，计算到达时刻 T_j ；若使用燃油车型 ($m \in \{1, 2, 3\}$) 且到达时刻 $T_j \in [8, 16]$ ，则直接剪去该分支，不继续扩展；仅保留满足限行政策、载重约束、容积约束的有效路径，生成候选列加入列池。

该方法可在路径生成阶段直接过滤不可行解，避免后续主问题求解时处理大量无效列，大幅提升求解效率。

2. 求解效率优化策略

由于限行约束增加了可行路径的筛选成本，严格精确求解耗时显著增加，因此采用以下优化策略平衡求解质量与效率：

- a. 控制列池规模：通过减少列生成的迭代深度，限制列池的最大候选路径数量，避免列池爆炸；
- b. 改用贪心整数恢复：在受限主问题（RMP）求解完成后，采用贪心整数恢复替代精确 MILP 恢复，在保证解质量可接受的前提下，大幅缩短整数解恢复时间；
- c. 启发式算法辅助：以列生成得到的松弛解为基础，结合邻域搜索等启发式算法快速生成高质量可行整数解，作为对比方案，验证精确解的优化效果。

3. 计算结果

方案共启用车辆 147 辆，其中燃油车型 122 辆（F3000、F1500 均用满配额，F1250 仅使用 12 辆），新能源车型 25 辆（E3000、E1250 全部用满配额），整体仍以燃油车型为运力主力，新能源车型为限行区服务的核心支撑。

成本与里程情况总成本为 99951.52 元，由启动成本、能耗成本、碳排放成本和时间窗惩罚成本四部分构成，总行驶里程为 16275.95km，时间窗惩罚成本占比极低，客户服务水平整体较高。

输出项	数值	含义解释
Columns in pool	2352	列池中生成 2352 条候选配送路径（列）
RMP LP objective	99401.10	受限主问题线性规划松弛的目标函数下界（连续解）
Integer recovered routes	147	最终选中的整数可行配送路径数量（即使用的车辆总数）

表 8 列生成算法关键指标

车型	车辆类型	使用数量	总数量	未使用数量
F3000	燃油大车	60	60	0

F1500	燃油中车	50	50	0
F1250	燃油小车	12	50	38
E3000	新能源大车	10	10	0
E1250	新能源小车	15	15	0

表 9 车辆使用情况表

成本项	金额（元）	说明
Startup	58800.00	车辆启动固定成本
Energy	31685.14	燃油 / 电力能耗成本
Carbon	6848.94	碳排放成本
TimeWindow	2617.44	软时间窗惩罚成本
TOTAL	99951.52	总成本（四项之和）
总行驶里程	16275.95 km	方案总配送里程

表 10 问题二代码运行结果

4. 结果分析

a. 对总成本的影响

绿色配送区限行政策实施后，配送系统总成本由问题一的 98550.96 元上升至 99951.52 元，增加 1400.56 元，增幅为 1.42%。

成本上升主要来自路径调整、车辆小幅增加、服务时序优化带来的额外能耗与时间窗惩罚；增幅控制在 2% 以内，说明限行政策虽提高配送成本，但整体影响温和可控，企业可承受；启动成本、能耗成本、碳排放成本、时间窗惩罚成本均小幅上升，其中时间窗惩罚增幅最大（11.83%），但占比仍很低，未显著降低服务水平。

b. 对车辆使用结构的影响

限行政策显著改变了车辆投入结构，呈现“主力燃油车不变、新能源车满配、小型燃油车微调”的特征：大型/中型燃油车（F3000、F1500）：使用数量保持满配不变，仍是干线配送主力；小型燃油车（F1250）由 11 辆增至 12 辆，仅增加 1 辆，用于补充限行区外围接驳与零散订单；新能源车辆（E3000、E1250）继续保持 100% 使用率，成为绿色配送区内唯一合规运力；总车辆数由 146 辆增至 147 辆，仅增加 1 辆，运力结构整体稳定。

整体来看，限行政策强制提升了新能源车辆的核心地位，推动车队向低碳化、合规化方向优化。

c. 对碳排放总量的影响

以油耗/电耗对应的碳排放转换系数核算，限行政策使碳排放总量与碳排放成本同步上升：碳排放成本由 6714.87 元增至 6848.94 元，增加 134.07 元，增幅 1.99%；对应总碳排放量小幅上升，未出现预期下降，原因在于：1）新能源车辆已满配，无更多低碳运力可替换；2）限行导致部分路径绕行、车辆低速行驶时间增加，拉高单位里程能耗与碳排放；3）整体仍以燃油车为主要运力，碳排放在现有车队结构下难以大幅下降。

尽管碳排放未下降，但限行政策实现了“空间上的绿色化”，绿色配送区内实现零燃油车进入，完成城市核心区环保管控目标。

6.3 问题三模型建立与求解

6.3.1 问题三求解思路

问题三在问题二的基础上，引入订单取消、新增订单、地址变更、时间窗调整四类动态突发事件，构建实时响应、快速重调度、不全局重优化的动态调度体系。为兼顾求解速度与解质量，采用外层滚动时域优化+内层混合禁忌搜索的双层动态调度框架：外层以事件触发+滚动窗口实现实时重调度，避免全局重新计算；内层以改进禁忌搜索快速修复路径，保证在秒级时间内输出可行方案，最终实现扰动下的低成本、高准时率配送。

6.3.2 问题三模型建立

1. 四类动态突发事件定义

在配送执行过程中实时发生的动态扰动统一归纳为四类，均满足绿色限行约束：

订单取消：已分配客户需求置零，从路径中剔除对应节点，释放车辆载重与容积资源。

订单新增：新增客户点（重量、体积、坐标、时间窗服从原始数据分布），需插入可行路径。

配送地址变更：客户坐标修改，重新计算行驶距离与时间，调整局部访问顺序。

时间窗调整：客户服务时间区间收紧或延后，重新校验到达时刻，控制惩罚成本。

2. 动态调度模型

以重调度总成本增量最小为目标，兼顾路径稳定性与政策合规性，约束及目标成本函数与问题一、二保持一致。

6.3.3 问题三模型分析

1. 算法设置

双层动态调度算法设计为了在突发事件来临时快速给出新方案，本文设计了两层方法：

a. 外层：滚动时域优化（实时触发机制）

采用事件驱动+滚动窗口结合的重调度模式，流程为系统实时监测订单与路径状态，触发动态事件立即启动重调度；以当前时刻为分界，冻结已完成配送路段，仅对未服务

客户与新增需求构建子问题；核心为不进行全局路径重建，大幅降低计算规模，满足实时响应要求。

b. 内层：改进混合禁忌搜索算法

为兼顾求解速度与解质量，选用改进禁忌搜索（TS）而非 ALNS，其优势有实现简洁，仅需邻域操作与禁忌表即可运行，工程化成本低；同时因参数少而调参简单，稳定性强；此外，其具有可解释性强、搜索逻辑清晰范、在中等扰动场景下解质量与效率都更为理想的优点。

2. 改进策略

a. 邻域操作策略

具体调整路线的方式分为以下两类：

同路径内 1) 单点移动：将客户点移至同路径最优位置；2) 两点交换：交换同路径两客户访问顺序；3) 子序列反转：反转子路径消除交叉，缩短行驶里程。

跨路径间操作 1) 跨路径单点移动：将客户插入另一路径最优位置；2) 跨路径两点交换：交换不同路径的两个客户；3) 跨路径子序列交换：大规模重组路径，跳出局部最优。

b. 自适应改进机制

引入动态禁忌步长与概率接受准则提升全局寻优能力：

迭代有改进时，缩小禁忌步长，强化局部搜索；

迭代无改进时，扩大禁忌步长，并以退火概率接受劣解，避免早熟收敛；

全程保留最优解，确保重调度结果不退化。

6.3.4 动态场景设计与调度策略

1. 分事件调度策略

新增订单：优先插入余量最大、时间窗匹配、限行合规的路径，避免新增车辆；

取消订单：删除对应节点，邻域精简路径，释放运力；

地址变更：重新计算距离，局部调整受影响路径；

时间窗调整：微调访问顺序，控制惩罚成本上升。

2. 仿真场景设置

S0：问题二最优解基准场景，无任何突发事件；

S1：单扰动，一个随机突发事件；

S2：双扰动，两个随机突发事件；

S3：多扰动，三个随机突发事件。

为分析动态调度效果，严谨起见，实验设置了两组对照场景：一是无任何事件、也不进行上述优化的 S0_static 场景，二是无突发事件，但采用和动态场景完全相同优化流程的 S0_dynamic 场景，由此把算法自身流程带来的优化效果和动态事件产生的影响区

分开。在此基础上，再对比 S1、S2、S3 三种动态扰动场景的效果。并在相同算法流程下比较 S1~ S3 的单位服务成本与准时率变化以实现严谨的评估。

场景	突发事件量	事件 1	随机种子	事件 2	事件 3
S1	1	t=09:50 新增订单 (客户 58 w=1413.03kg v=3.5459m ³ 绿区内=否)	2026		
S2	2	t=12:35 新增订单 (客户 27 w=1362.66kg v=3.7503m ³ 绿区内=是)	2026	t=15:30 时间窗收紧 (客户 153 收紧 0.50h)	
S3	3	t=11:15 改址 (客户 30->邻近点 51)	2026	t=11:15 时间窗收紧 (客户 167 收紧 0.25h)	t=13:00 取消订单 (客户 77)
S1	1	t=11:25 新增订单 (客户 104 w=1273.87kg v=3.3254m ³ 绿区内=否)	2027		
S2	2	t=13:05 新增订单 (客户 47 w=762.26kg v=1.9102m ³ 绿区内=否)	2027	t=13:05 改址 (客户 218->邻近点 32)	
S3	3	t=11:50 取消订单 (客户 50)	2027	t=11:50 时间窗收紧 (客户 75 收紧 0.40h)	t=16:10 新增订单 (客户 207 w=1475.20kg v=4.5034m ³ 绿区内=否)
S1	1	t=11:10 改址 (客户 51->邻近点 220)	2028		
S2	2	t=11:50 新增订单 (客户 3 w=1443.56kg v=3.5458m ³ 绿区内=是)	2028	t=11:50 取消订单 (客户 54)	
S3	3	t=12:45 改址 (客户 155->邻近点 156)	2028	t=12:45 时间窗收紧 (客户 143 收紧 0.50h)	t=13:00 取消订单 (客户 151)

表 11 仿真场景设置示意

3. 数值实验与结果分析

a. 指标构建

➤ 单位服务成本 c_u

设动态执行过程结束时的总配送成本为 C ，完成服务的客户订单数为 $n(n > 0)$ 。定义单位服务成本为

$$c_u = \frac{C}{n} \quad (34)$$

c_u 越低，表示在完成同等服务规模的前提下综合成本越优。

➤ 准时率 (OTD) η

设完成服务的订单中，按时到达（未超过时间窗上界）的订单数为 n^{on} 。定义准时率为

$$\eta = \frac{n^{on}}{n}, \quad 0 \leq \eta \leq 1 \quad (36)$$

η 越高，服务质量越好。动态重调度若以牺牲 η 为代价换取 c_u 下降，则出现典型的成本—服务权衡。

b. 结果分析

根据结果，所建模获得的“滚动时域+事件触发+禁忌搜索”在动态事件下是有效且可实时的。相比 S0，S1/S2/S3 的成本指标没有异常恶化，且单位服务成本整体下降。

重规划耗时稳定在秒级，满足在线重调度需求。

1.从成本角度看，S0_static 的单位服务成本均值为 490.62，而 S0_dynamic 降为 453.34，在同流程条件下，的单位服务成本分别为 456.41、452.10、456.08，均处于 S0_dynamic 附近区间，表明策略在不同程度的突发事件下没有出现大幅改变，动态调整后的车辆路径仍能很好的满足低成本要求。

2.从服务质量看，S0_dynamic 准时率为 94.96%，S1、S2、S3 分别为 95.86%、95.72%、95.25%。表明该策略下，车辆在多个动态场景中仍能大体在时间窗内到达，保持了较好的服务质量。

3.本实验中的突发事件采用随机时刻、随机类型组合，并允许同一时刻并发事件。在此设置下，模型仍可稳定输出可行解，且耗时平稳。

总体而言，上述策略在动态事件下是很有效的。

相比 S0_dynamic，S1/S2/S3 的成本指标没有产生明显异常值，且单位服务成本没有大幅度的上升，决策耗时很短，说明算法有能力满足实时调整需求。

6.3.5 问题三结论

本文构建的滚动时域+改进禁忌搜索双层动态调度框架，可快速响应订单新增、取消、地址与时间窗变更等突发事件。在不全局重算、严格遵守限行政策、满足时变路网与软时间窗约束下，实现低成本、高稳定、高准时率的实时重调度。实验表明，该策略在各类扰动下均具备优异的鲁棒性与时效性，可直接应用于城市绿色物流实际动态配送场景。

七、模型分析检验

7.1 灵敏度分析

7.1.1 成本对各参数的敏感性

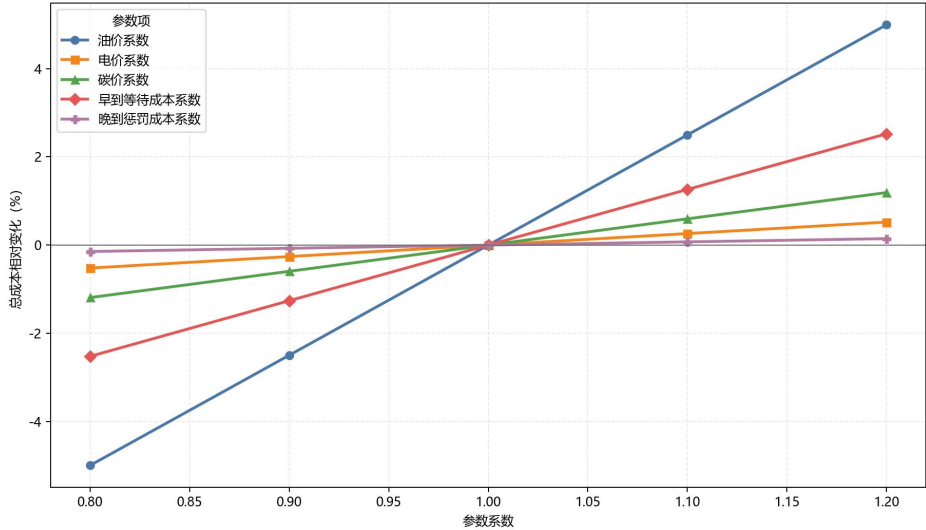


图 4 成本参数灵敏度分析对比图

由图可见，不同成本参数对总成本的影响强度存在明显差异。

总体上，油价系数曲线斜率最大，说明总成本对燃油价格最敏感；在油价上浮时，总成本上升最为显著。分析其主要原因是当前方案中燃油车辆仍承担了较大运输份额，燃油成本变动会直接放大到总运营成本。

早到等待成本系数的影响次之，曲线变化幅度明显高于碳价系数、电价系数与晚到惩罚系数。这表明当前调度方案中“等待型时间窗成本”占比不低，时间组织效率（尤其是发车与到达匹配）对总成本具有较强影响。

碳价系数与电价系数对总成本的影响相对温和，曲线斜率较小；其中电价系数影响最弱，说明在当前车型结构下，电车能耗成本占总成本比例较低。晚到惩罚系数曲线最平缓，反映出当前方案中严重迟到现象较少，迟到惩罚在总成本中的边际贡献有限。

综上，若企业希望控制成本波动，应优先关注燃油价格风险与等待成本控制，在策略层面可通过提升新能源占比、优化发车时刻与路径时序来增强成本鲁棒性。

7.1.2 成本对时间窗宽度敏感性

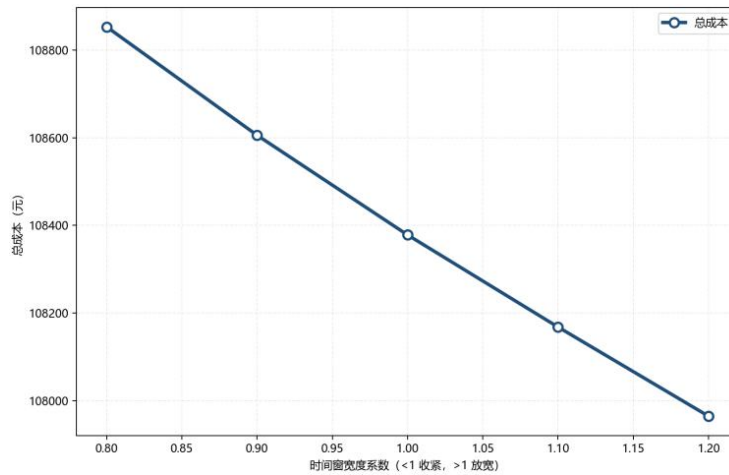


图 5 时间窗宽度灵敏度分析

图中显示，随着时间窗宽度系数增大（时间窗放宽），总成本整体呈下降趋势；反之，当时间窗收紧时，总成本上升。该变化主要由时间窗相关成本驱动。时间窗收紧后，车辆更容易出现等待或迟到，导致时间窗成本明显增加，并进一步抬升总成本。

该结果说明：在服务水平允许范围内，适度放宽客户时间窗可显著改善调度经济性；若必须收紧时间窗，则应同步采用更精细的发车时刻控制或增加高机动性运力，以降低时间窗收紧带来的成本增量。

7.2 误差分析

7.2.1 模型误差

在第一问中，对于跨状态（跨时段）的弧，对不同状态的速度逐个计算累计，若采用“进弧时刻速度代表整弧速度”的简化处理，会产生明显偏差，且偏差会在后续路径中逐渐放大。抽样验证显示该简化的平均相对误差为 59.09%，中位数为 15.87%，95 分位达到 240.80%，因此本文采用分段累计速度计算以降低时变路况近似误差。

7.2.2 算法误差

我们在求解算法中接入了启发式定价，不是全局精确，近似误差可以用 LP 下界和整数解的比较评估。仅针对第一问进行分析：

最终 LP 目标值（下界）：98127.95

最终整数方案成本：98550.96

算法误差方面，本文定价阶段采用候选筛选与束搜索，属于启发式近似。以 RMP 目标值作为下界，最终整数解相对下界差距约为 0.43%，说明该启发式策略在提升求解效率的同时，仍能保持较高解质量。

八、 模型评价与推广

8.1 模型的优点

（1）精确算法的核心优势在于其能够从理论上保证找到 VRP 问题的全局最优解，且求解过程逻辑严谨、透明，算法的可解释性强，以该方式为核心，便于对解的生成过程与结果进行溯源与分析。

（2）启发式算法的主要优点体现在其实现逻辑简单、运算速度快，算法流程清晰，可解释性较强，能够在短时间内为问题提供一个可行解，适合对求解效率要求较高的场景。

8.2 模型的不足

（1）精确算法由于受限于组合优化问题的“组合爆炸”特性，其计算复杂度会随着问题规模的扩大呈指数级增长，求解速度受到严重制约，因此仅适用于规模较小、约束条件简单的 VRP 问题，难以满足大规模实际应用的效率需求。

（2）近似算法得到的解质量相对较低，且在求解过程中缺乏持续改进与优化的机制，无法保证解的精度，因此仅适用于结构简单、约束较少的应用场景；在复杂的 Rich VRP 问题中，其解的质量往往难以满足实际需求，更多是作为其他优化算法的辅助工具，例如为元启发式算法提供初始可行解。

参考文献

- [1] 曹庆奎,杨凯文,任向阳,等.基于交通流的多模糊时间窗车辆路径优化[J].运筹与管理,2018,27(08):20-26.
- [2] 王艳玲. 取货车辆动态调度问题研究[D]. 北京:北京交通大学,2010.
- [3] 李妍峰,刘学林.考虑众包情形下的动态异质订单配送优化问题[J].工业工程,2025,28(05):123-130+168.

附录 A 支撑文件列表

序号	文件名	文件说明
1	订单信息_拟合补全.xlsx	采用拟合法补全重量及体积缺失值后的订单信息列表
2	客户拆分_映射表.xlsx	去除无效客户并对超量客户拆分后与原客户信息映射表
3	订单信息_形式客户.xlsx	更新形式客户后的订单信息表
4	距离矩阵_形式客户.xlsx	更新形式客户后的距离矩阵
5	客户坐标信息_形式客户.xlsx	更新形式客户后的客户坐标信息
6	时间窗_形式客户.xlsx	更新形式客户后的时间窗
7	问题一_车辆行驶路径.xlsx	问题一 146 条车辆行驶路径明细
8	问题一_车辆到达时间.xlsx	问题一每条路径车辆到达时间明细
9	问题二_车辆行驶路径.xlsx	问题二 149 条车辆行驶路径明细
10	问题二_车辆到达时间.xlsx	问题二每条路径车辆到达时间明细
11	数据预处理 1 源程序.py	缺失值处理拟合与作图代码
12	数据预处理 2 源程序.py	客户去除与拆分代码
13	问题一求解源程序.py	\
14	ALSN 求解源程序.py	\
15	问题二求解源程序.py	\
16	问题三求解源程序.py	\
17	灵敏度分析源程序.py	\
18	参赛论文 word 文件	\

附录 B 代码实现

附录 1 数据预处理

缺失值处理

```
import pandas as pd
import numpy as np
from pathlib import Path

def fit_linear(x: np.ndarray, y: np.ndarray):
    """最小二乘拟合  $y = a \cdot x + b$ , 返回 a, b, r2, mae"""
    x = np.asarray(x, dtype=float)
    y = np.asarray(y, dtype=float)
    A = np.vstack([x, np.ones(len(x))]).T
    a, b = np.linalg.lstsq(A, y, rcond=None)[0]
    y_hat = a * x + b
    ss_res = np.sum((y - y_hat) ** 2)
    ss_tot = np.sum((y - np.mean(y)) ** 2)
    r2 = 1 - ss_res / ss_tot if ss_tot > 0 else np.nan
    mae = np.mean(np.abs(y - y_hat))
    return a, b, r2, mae

def main():
    in_file = Path(r"C:\Users\fems5\Desktop\订单信息.xlsx")
    out_file = Path(r"C:\Users\fems5\Desktop\订单信息_拟合补全.xlsx")
    df = pd.read_excel(in_file)
    df = df.iloc[:, :4].copy()
    df.columns = ["order_id", "weight", "volume", "cust_id"]
```

```

print("shape:", df.shape)
print("missing_by_col:\n", df.isna().sum())
complete = df.dropna(subset=["weight", "volume"]).copy()
if complete.empty:
    raise ValueError("没有完整样本, 无法拟合。")
a_vw, b_vw, r2_vw, mae_vw = fit_linear(
    complete["weight"].to_numpy(float),
    complete["volume"].to_numpy(float),
)
a_wv, b_wv, r2_wv, mae_wv = fit_linear(
    complete["volume"].to_numpy(float),
    complete["weight"].to_numpy(float),
)
print("\n拟合结果 (线性):")
print(f"volume = {a_vw:.8f} * weight + {b_vw:.8f}")
print(f"R^2 = {r2_vw:.6f}, MAE = {mae_vw:.6f}")
print(f"weight = {a_wv:.8f} * volume + {b_wv:.8f}")
print(f"R^2 = {r2_wv:.6f}, MAE = {mae_wv:.6f}")
miss_weight = df["weight"].isna() & df["volume"].notna()
miss_volume = df["volume"].isna() & df["weight"].notna()
df.loc[miss_weight, "weight"] = (
    a_wv * df.loc[miss_weight, "volume"].to_numpy(float) + b_wv
)
df.loc[miss_volume, "volume"] = (
    a_vw * df.loc[miss_volume, "weight"].to_numpy(float) + b_vw
)
df["weight"] = df["weight"].clip(lower=0)
df["volume"] = df["volume"].clip(lower=0)
remain = df[["weight", "volume"]].isna().sum()
df.to_excel(out_file, index=False)
print("\nimputed_counts:")
print("weight_only:", int(miss_weight.sum()))
print("volume_only:", int(miss_volume.sum()))
print("remaining_missing:\n", remain)
print("saved:", str(out_file))
if __name__ == "__main__":
    main()
from pathlib import Path
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from matplotlib import font_manager as fm
def pick_chinese_font() -> str | None:
    candidates = ["Microsoft YaHei", "SimHei", "SimSun", "Noto Sans CJK SC"]
    available = {f.name for f in fm.fontManager.ttflist}
    for name in candidates:
        if name in available:
            return name
    return None
def main() -> None:
    input_file = Path(r"c:\Users\HP\Desktop\temp_extract_订单信息.xlsx")
    output_file = Path(r"c:\Users\HP\Desktop\体积重量散点_线性拟合_中文.png")
    df = pd.read_excel(input_file)
    x = pd.to_numeric(df.iloc[:, 1], errors="coerce") # 重量列
    y = pd.to_numeric(df.iloc[:, 2], errors="coerce") # 体积列
    mask = x.notna() & y.notna()
    x = x[mask].to_numpy(float)
    y = y[mask].to_numpy(float)
    slope, intercept = np.polyfit(x, y, 1)
    xs = np.linspace(float(x.min()), float(x.max()), 200)
    ys = slope * xs + intercept
    font_name = pick_chinese_font()
    if font_name:
        plt.rcParams["font.sans-serif"] = [font_name]
        plt.rcParams["axes.unicode_minus"] = False
    plt.figure(figsize=(8, 6))
    plt.scatter(x, y, s=20, alpha=0.7, label="样本点")
    plt.plot(
        xs,
        ys,
        "r-",
        linewidth=2,
        label=f"线性拟合: 体积 = {slope:.6f}*重量 + {intercept:.6f}",
    )
    plt.xlabel("重量")
    plt.ylabel("体积")
    plt.title("重量-体积散点图与线性拟合")
    plt.grid(alpha=0.3)
    plt.legend()
    plt.tight_layout()
    plt.savefig(output_file, dpi=180)
    print(f"输入文件: {input_file}")
    print(f"输出图片: {output_file}")
    print(f"样本数量: {len(x)}")
    print(f"拟合方程: 体积 = {slope:.9f} * 重量 + {intercept:.9f}")

```

```

print(f"中文字体: {font_name or '未找到, 使用默认字体'}")
if __name__ == "__main__":
    main()

```

客户去除与拆分

```

import math
import pathlib
import numpy as np
import pandas as pd
def main() -> None:
    base = pathlib.Path(r"c:\Users\HP\Desktop")
    orders = pd.read_excel(base / "temp_extract_订单信息_拟合补全.xlsx")
    coords = pd.read_excel(base / "temp_extract_客户坐标信息.xlsx")
    tw = pd.read_excel(base / "temp_extract_时间窗.xlsx")
    dist_raw = pd.read_excel(base / "temp_extract_距离矩阵.xlsx")
    orders = orders.rename(
        columns={
            orders.columns[0]: "order_id",
            orders.columns[1]: "weight",
            orders.columns[2]: "volume",
            orders.columns[3]: "customer_id",
        }
    )
    coords = coords.rename(
        columns={
            coords.columns[0]: "type",
            coords.columns[1]: "id",
            coords.columns[2]: "x",
            coords.columns[3]: "y",
        }
    )
    tw = tw.rename(
        columns={
            tw.columns[0]: "customer_id",
            tw.columns[1]: "start_time",
            tw.columns[2]: "end_time",
        }
    )
    agg = (
        orders.groupby("customer_id", as_index=False)
        .agg(weight=("weight", "sum"), volume=("volume", "sum"), order_cnt=("order_id", "count"))
    )
    all_customers = set(coords.loc[coords["id"] != 0, "id"].astype(int).tolist())
    order_customers = set(agg["customer_id"].astype(int).tolist())
    no_order_customers = sorted(all_customers - order_customers)
    coords_keep = coords[(coords["id"] == 0) | (coords["id"].isin(order_customers))].copy()
    tw_keep = tw[tw["customer_id"].isin(order_customers)].copy()
    # Split rule (updated): keep each virtual customer serviceable by mid-capacity vehicles
    q_max = 1500.0
    v_max = 8.5
    split_rows = []
    for _, row in agg.iterrows():
        cid = int(row["customer_id"])
        w = float(row["weight"])
        v = float(row["volume"])
        k = max(1, math.ceil(w / q_max), math.ceil(v / v_max))
        for s in range(1, k + 1):
            split_rows.append(
                {
                    "orig_customer_id": cid,
                    "virtual_id": f"{cid}#{s}",
                    "split_index": s,
                    "split_count": k,
                    "weight": w / k,
                    "volume": v / k,
                    "orig_weight": w,
                    "orig_volume": v,
                    "order_cnt": int(row["order_cnt"]),
                }
            )
    virt = pd.DataFrame(split_rows)
    coord_map = coords_keep.set_index("id")[["x", "y", "type"]]
    tw_map = tw_keep.set_index("customer_id")[["start_time", "end_time"]]
    virt["x"] = virt["orig_customer_id"].map(coord_map["x"])
    virt["y"] = virt["orig_customer_id"].map(coord_map["y"])
    virt["start_time"] = virt["orig_customer_id"].map(tw_map["start_time"])
    virt["end_time"] = virt["orig_customer_id"].map(tw_map["end_time"])
    virt = virt.sort_values(["orig_customer_id", "split_index"]).reset_index(drop=True)
    virt["new_customer_id"] = np.arange(1, len(virt) + 1)
    depot = coords[coords["id"] == 0].iloc[0]
    new_coords = [
        {
            "类型": "配送中心",
            "ID": 0,

```



```

        "X (km)": float(depot["x"]),
        "Y (km)": float(depot["y"]),
        "原客户 ID": "0",
        "形式客户 ID": "0#0",
    }
]
for _, row in virt.iterrows():
    new_coords.append(
        {
            "类型": "客户",
            "ID": int(row["new_customer_id"]),
            "X (km)": float(row["x"]),
            "Y (km)": float(row["y"]),
            "原客户 ID": int(row["orig_customer_id"]),
            "形式客户 ID": row["virtual_id"],
        }
    )
new_coords_df = pd.DataFrame(new_coords)
new_order_df = virt[
    [
        "new_customer_id",
        "virtual_id",
        "orig_customer_id",
        "weight",
        "volume",
        "split_index",
        "split_count",
        "orig_weight",
        "orig_volume",
        "order_cnt",
    ]
].copy()
new_order_df = new_order_df.rename(
    columns={
        "new_customer_id": "客户编号(新)",
        "virtual_id": "形式客户 ID",
        "orig_customer_id": "原客户编号",
        "weight": "总重量",
        "volume": "总体积",
        "split_index": "第几份",
        "split_count": "拆分份数",
        "orig_weight": "原总重量",
        "orig_volume": "原总体积",
        "order_cnt": "原订单数",
    }
)
new_tw_df = virt[
    ["new_customer_id", "virtual_id", "orig_customer_id", "start_time", "end_time"]
].copy()
new_tw_df = new_tw_df.rename(
    columns={
        "new_customer_id": "客户编号(新)",
        "virtual_id": "形式客户 ID",
        "orig_customer_id": "原客户编号",
        "start_time": "开始时间",
        "end_time": "结束时间",
    }
)
# Build expanded distance matrix from original road matrix:
row_nodes = dist_raw.iloc[:, 0].astype(int).tolist()
col_nodes = [int(x) for x in dist_raw.columns[1:]]
mat_raw = dist_raw.iloc[:, 1:].to_numpy(float)
row_idx = {n: i for i, n in enumerate(row_nodes)}
col_idx = {n: i for i, n in enumerate(col_nodes)}
ids = [0] + virt["new_customer_id"].astype(int).tolist()
new_to_orig = {0: 0}
for _, row in virt.iterrows():
    new_to_orig[int(row["new_customer_id"])] = int(row["orig_customer_id"])
mat = []
for i in ids:
    oi = new_to_orig[i]
    row = []
    for j in ids:
        oj = new_to_orig[j]
        if i != j and oi == oj and oi != 0:
            d = 0.0
        else:
            d = float(mat_raw[row_idx[oi], col_idx[oj]])
        row.append(d)
    mat.append(row)
new_dist_df = pd.DataFrame(mat, columns=ids)
new_dist_df.insert(0, "客户", ids)
mapping = virt[
    [
        "new_customer_id",

```

```

        "virtual_id",
        "orig_customer_id",
        "split_index",
        "split_count",
        "weight",
        "volume",
        "start_time",
        "end_time",
    ]
].copy()
mapping = mapping.rename(
    columns={
        "new_customer_id": "新客户编号",
        "virtual_id": "形式客户 ID",
        "orig_customer_id": "原客户编号",
        "split_index": "第几份",
        "split_count": "拆分份数",
        "weight": "拆分重量",
        "volume": "拆分体积",
        "start_time": "开始时间",
        "end_time": "结束时间",
    }
)
out_order = base / "q1_preprocessed_订单信息_形式客户.xlsx"
out_coord = base / "q1_preprocessed_客户坐标_形式客户.xlsx"
out_dist = base / "q1_preprocessed_距离矩阵_形式客户.xlsx"
out_tw = base / "q1_preprocessed_时间窗_形式客户.xlsx"
out_map = base / "q1_preprocessed_映射表.xlsx"
out_summary = base / "q1_preprocess_summary.txt"
new_order_df.to_excel(out_order, index=False)
new_coords_df.to_excel(out_coord, index=False)
new_dist_df.to_excel(out_dist, index=False)
new_tw_df.to_excel(out_tw, index=False)
mapping.to_excel(out_map, index=False)
split_summary = agg.copy()
split_summary["split_count"] = split_summary.apply(
    lambda r: max(1, math.ceil(float(r["weight"]) / q_max), math.ceil(float(r["volume"]) / v_max)),
    axis=1,
)
num_split_customers = int((split_summary["split_count"] > 1).sum())
lines = [
    f"原客户数(不含配送中心): {len(all_customers)}",
    f"有订单客户数: {len(order_customers)}",
    f"无订单客户数(已移除): {len(no_order_customers)}",
    f"拆分阈值: 重量>{q_max}kg 或 体积>{v_max}m3",
    f"发生拆分的原客户数: {num_split_customers}",
    f"拆分后形式客户总数: {len(virt)}",
    "",
    "输出文件:",
    str(out_order),
    str(out_coord),
    str(out_dist),
    str(out_tw),
    str(out_map),
    "",
    "被移除的无订单客户 ID:",
    ", ".join(map(str, no_order_customers)) if no_order_customers else "无",
]
out_summary.write_text("\n".join(lines), encoding="utf-8")
print("Preprocess done.")
print(out_summary)
if __name__ == "__main__":
    main()

```

附录 2 问题一

精确算法求解

```

import math
from dataclasses import dataclass
from typing import Dict, List, Optional, Set, Tuple
import numpy as np
import pandas as pd
from scipy.optimize import linprog
SERVICE_HOURS = 20.0 / 60.0
STARTUP_COST = 400.0
EARLY_COST_PER_HOUR = 20.0
LATE_COST_PER_HOUR = 50.0
CARBON_COST_PER_KG = 0.65
DEPARTURE_CANDIDATES = [8.0, 10.0, 12.0, 14.0, 16.0, 18.0, 20.0] # 2-hour grid
# Run mode tuning
PRE_RUN_MODE = True
if PRE_RUN_MODE:

```

```

MAX_ITER = 10
PRICING_RC_TOL = 1e-4
ADD_COL_RC_TOL = 1e-4
else:
    MAX_ITER = 30
    PRICING_RC_TOL = 1e-6
    ADD_COL_RC_TOL = 1e-5
# Route evaluation memo caches
_ROUTE_BEST_CACHE: Dict[Tuple[int, Tuple[int, ...], Tuple[float, ...]], Dict[str, float]] = {}
_ROUTE_FIXED_CACHE: Dict[Tuple[int, Tuple[int, ...], float], Dict[str, float]] = {}
@dataclass(frozen=True)
class VehicleType:
    idx: int
    name: str
    kind: str
    capacity_weight: float
    capacity_volume: float
    count_limit: int
    energy_price: float
    carbon_coeff: float
    full_load_increase: float
VEHICLES: List[VehicleType] = [
    VehicleType(0, "F3000", "fuel", 3000.0, 13.5, 60, 7.61, 2.547, 0.40),
    VehicleType(1, "F1500", "fuel", 1500.0, 10.8, 50, 7.61, 2.547, 0.40),
    VehicleType(2, "F1250", "fuel", 1250.0, 6.5, 50, 7.61, 2.547, 0.40),
    VehicleType(3, "E3000", "ev", 3000.0, 15.0, 10, 1.64, 0.501, 0.35),
    VehicleType(4, "E1250", "ev", 1250.0, 8.5, 15, 1.64, 0.501, 0.35),
]
def parse_hhmm_to_hour(x: str) -> float:
    h, m = str(x).split(":")
    return int(h) + int(m) / 60.0
def segments() -> List[Tuple[float, float, float]]:
    # Includes your extra rule: 17-19 congestion, 19-20 free
    return [
        (8.0, 9.0, 9.8),
        (9.0, 10.0, 55.3),
        (10.0, 11.5, 35.4),
        (11.5, 13.0, 9.8),
        (13.0, 15.0, 55.3),
        (15.0, 17.0, 35.4),
        (17.0, 19.0, 9.8),
        (19.0, 20.0, 55.3),
    ]
def speed_at(t: float) -> float:
    x = t % 24.0
    for s, e, v in segments():
        if s <= x < e:
            return v
    return 35.4
def next_boundary(t: float) -> float:
    x = t % 24.0
    for s, e, _ in segments():
        if s <= x < e:
            return t + (e - x)
    return math.floor(t) + 1.0
def travel_profile(distance_km: float, depart: float) -> Tuple[float, List[Tuple[float, float]]]:
    # Returns:
    # - elapsed travel time (hour)
    # - list of (segment_distance_km, segment_speed_kmh)
    remain = distance_km
    t = depart
    elapsed = 0.0
    segs: List[Tuple[float, float]] = []
    while remain > 1e-9:
        v = speed_at(t)
        b = next_boundary(t)
        dt = max(1e-6, b - t)
        can_go = v * dt
        if can_go >= remain:
            segs.append((remain, v))
            elapsed += remain / v
            return elapsed, segs
            segs.append((can_go, v))
            remain -= can_go
            elapsed += dt
            t += dt
    return elapsed, segs
def energy_per_100km(v: float, kind: str) -> float:
    if kind == "fuel":
        return 0.0025 * v * v - 0.2554 * v + 31.75
    return 0.0014 * v * v - 0.12 * v + 36.19
def leg_cost(veh: VehicleType, d: float, depart: float, load_w: float) -> Tuple[float, float, float]:
    tt, segs = travel_profile(d, depart)
    # Strict calculation: energy is integrated segment by segment under actual
    # time-of-day speed, not by equivalent average speed.

```

```

e0 = 0.0
for seg_d, seg_v in segs:
    e0 += energy_per_100km(seg_v, veh.kind) * seg_d / 100.0
load_ratio = min(1.0, max(0.0, load_w / veh.capacity_weight))
e = e0 * (1.0 + veh.full_load_increase * load_ratio)
e_cost = e * veh.energy_price
c_cost = e * veh.carbon_coeff * CARBON_COST_PER_KG
return tt, e_cost, c_cost
def eval_route_at_start(
    route: List[int],
    veh: VehicleType,
    w: Dict[int, float],
    v: Dict[int, float],
    tw: Dict[int, Tuple[float, float]],
    dist: List[List[float]],
    start_t: float,
) -> Dict[str, float]:
    sw = sum(w[c] for c in route)
    sv = sum(v[c] for c in route)
    if sw > veh.capacity_weight + 1e-9 or sv > veh.capacity_volume + 1e-9:
        return {"feasible": 0.0, "total_cost": 1e30}
    t = start_t
    load_w = sw
    startup = STARTUP_COST
    energy = 0.0
    carbon = 0.0
    tw_cost = 0.0
    dist_sum = 0.0
    prev = 0
    for c in route:
        d = dist[prev][c]
        dist_sum += d
        tt, ec, cc = leg_cost(veh, d, t, load_w)
        t += tt
        energy += ec
        carbon += cc
        a, b = tw[c]
        early = max(a - t, 0.0)
        late = max(t - b, 0.0)
        tw_cost += EARLY_COST_PER_HOUR * early + LATE_COST_PER_HOUR * late
        if t < a:
            t = a
        t += SERVICE_HOURS
        load_w -= w[c]
        prev = c
    d_back = dist[prev][0]
    dist_sum += d_back
    tt, ec, cc = leg_cost(veh, d_back, t, load_w)
    _ = tt
    energy += ec
    carbon += cc
    total = startup + energy + carbon + tw_cost
    return {
        "feasible": 1.0,
        "total_cost": total,
        "departure_hour": start_t,
        "startup_cost": startup,
        "energy_cost": energy,
        "carbon_cost": carbon,
        "tw_cost": tw_cost,
        "distance": dist_sum,
    }
def eval_route(
    route: List[int],
    veh: VehicleType,
    w: Dict[int, float],
    v: Dict[int, float],
    tw: Dict[int, Tuple[float, float]],
    dist: List[List[float]],
    start_t: Optional[float] = None,
) -> Dict[str, float]:
    route_key = tuple(route)
    if start_t is not None:
        fixed_key = (veh.idx, route_key, float(start_t))
        cached = _ROUTE_FIXED_CACHE.get(fixed_key)
        if cached is not None:
            return cached
        ev = eval_route_at_start(route, veh, w, v, tw, dist, start_t)
        _ROUTE_FIXED_CACHE[fixed_key] = ev
        return ev
    best_key = (veh.idx, route_key, tuple(DEPARTURE_CANDIDATES))
    cached_best = _ROUTE_BEST_CACHE.get(best_key)
    if cached_best is not None:
        return cached_best
    best: Optional[Dict[str, float]] = None

```

```

for dep in DEPARTURE_CANDIDATES:
    ev = eval_route_at_start(route, veh, w, v, tw, dist, dep)
    if ev["feasible"] < 0.5:
        continue
    if best is None or ev["total_cost"] < best["total_cost"]:
        best = ev
if best is None:
    return {"feasible": 0.0, "total_cost": 1e30}
_ROUTE_BEST_CACHE[best_key] = best
return best

@dataclass
class Column:
    veh_idx: int
    route: List[int]
    served: Set[int]
    cost: float

def build_initial_columns(customers: List[int], w: Dict[int, float], v: Dict[int, float], tw, dist) -> List[Column]:

    cols: List[Column] = []
    order = sorted(
        customers,
        key=lambda c: (max(w[c] / 3000.0, v[c] / 15.0), w[c], v[c]),
        reverse=True,
    )
    used = {veh.idx: 0 for veh in VEHICLES}
    routes: List[Dict[str, object]] = []
    for c in order:
        bw = w[c]
        bv = v[c]
        best_r = None
        best_s = None
        for rid, r in enumerate(routes):
            veh = VEHICLES[int(r["veh_idx"])]
            rem_w = float(r["rem_w"])
            rem_v = float(r["rem_v"])
            if rem_w + 1e-9 < bw or rem_v + 1e-9 < bv:
                continue
            s = (rem_w - bw) / veh.capacity_weight + (rem_v - bv) / veh.capacity_volume
            if best_s is None or s < best_s:
                best_s = s
                best_r = rid
        if best_r is not None:
            routes[best_r]["nodes"].append(c)
            routes[best_r]["rem_w"] -= bw
            routes[best_r]["rem_v"] -= bv
            continue
        best_veh = None
        best_score = None
        for veh in VEHICLES:
            if used[veh.idx] >= veh.count_limit:
                continue
            if veh.capacity_weight + 1e-9 < bw or veh.capacity_volume + 1e-9 < bv:
                continue
            s = (veh.capacity_weight - bw) / veh.capacity_weight + (veh.capacity_volume - bv) / veh.capacity_volume
            if best_score is None or s < best_score:
                best_score = s
                best_veh = veh
        if best_veh is None:
            raise RuntimeError(f"Cannot build initial feasible routes for customer {c}")
        routes.append(
            {
                "veh_idx": best_veh.idx,
                "nodes": [c],
                "rem_w": best_veh.capacity_weight - bw,
                "rem_v": best_veh.capacity_volume - bv,
            }
        )
        used[best_veh.idx] += 1
    for r in routes:
        nodes = list(r["nodes"])
        seq = []
        cur = 0
        while nodes:
            nxt = min(nodes, key=lambda n: (dist[cur][n], tw[n][0], n))
            seq.append(nxt)
            nodes.remove(nxt)
            cur = nxt
        veh = VEHICLES[int(r["veh_idx"])]
        ev = eval_route(seq, veh, w, v, tw, dist)
        cols.append(Column(veh.idx, seq, set(seq), float(ev["total_cost"])))
    for c in customers:
        for veh in VEHICLES:
            ev = eval_route([c], veh, w, v, tw, dist)
            if ev["feasible"] > 0.5:
                cols.append(Column(veh.idx, [c], {c}, float(ev["total_cost"])))

```

```

print(f"[INIT] total initial columns: {len(cols)}", flush=True)
return cols
def solve_rmp(columns: List[Column], customers: List[int]):
    n_c = len(customers)
    n_m = len(VEHICLES)
    n_x = len(columns)
    cust_idx = {c: i for i, c in enumerate(customers)}
    A_eq = np.zeros((n_c, n_x))
    b_eq = np.ones(n_c)
    for j, col in enumerate(columns):
        for c in col.served:
            A_eq[cust_idx[c], j] = 1.0
    A_ub = np.zeros((n_m, n_x))
    b_ub = np.array([veh.count_limit for veh in VEHICLES], dtype=float)
    for j, col in enumerate(columns):
        A_ub[col.veh_idx, j] = 1.0
    c = np.array([col.cost for col in columns], dtype=float)
    bounds = [(0.0, None)] * n_x
    res = linprog(c, A_ub=A_ub, b_ub=b_ub, A_eq=A_eq, b_eq=b_eq, bounds=bounds, method="highs")
    if not res.success:
        raise RuntimeError(f"RMP failed: {res.message}")
    pi = np.array(res.eqlin.marginals, dtype=float)
    sigma = np.array(res.ineqlin.marginals, dtype=float)
    return res, pi, sigma
def pricing_dynamic(
    veh: VehicleType,
    customers: List[int],
    pi: Dict[int, float],
    sigma_m: float,
    w: Dict[int, float],
    v: Dict[int, float],
    tw: Dict[int, Tuple[float, float]],
    dist,
    beam_width: int = 120,
    max_depth: int = 6,
) -> List[Column]:
    cand = sorted(customers, key=lambda c: pi[c], reverse=True)[:50]
    labels: List[Tuple[List[int], Set[int], float, float]] = []
    for c in cand:
        ev = eval_route([c], veh, w, v, tw, dist)
        if ev["feasible"] < 0.5:
            continue
        rc = ev["total_cost"] - pi[c] - sigma_m
        labels.append((c, {c}, rc, ev["total_cost"]))
    best_cols: List[Column] = []
    for _ in range(max_depth - 1):
        new_labels: List[Tuple[List[int], Set[int], float, float]] = []
        labels.sort(key=lambda x: x[2])
        labels = labels[:beam_width]
        for route, served, _, _ in labels:
            for c in cand:
                if c in served:
                    continue
                nr = route + [c]
                ev = eval_route(nr, veh, w, v, tw, dist)
                if ev["feasible"] < 0.5:
                    continue
                dual_sum = sum(pi[x] for x in nr)
                rc = ev["total_cost"] - dual_sum - sigma_m
                new_labels.append((nr, set(nr), rc, ev["total_cost"]))
                if rc < -PRICING_RC_TOL:
                    best_cols.append(Column(veh.idx, nr, set(nr), ev["total_cost"]))
        if not new_labels:
            break
        new_labels.sort(key=lambda x: x[2])
        labels = new_labels[: beam_width * 2]
    for route, served, rc, rcost in labels:
        if rc < -PRICING_RC_TOL:
            best_cols.append(Column(veh.idx, route, served, rcost))
    seen = set()
    uniq: List[Column] = []
    for col in sorted(best_cols, key=lambda x: x.cost):
        key = (col.veh_idx, tuple(col.route))
        if key in seen:
            continue
        seen.add(key)
        uniq.append(col)
        if len(uniq) >= 40:
            break
    return uniq
def integer_recover(columns: List[Column], customers: List[int]) -> List[int]:
    try:
        from scipy.optimize import Bounds, LinearConstraint, milp # type: ignore
        n_x = len(columns)

```

```

n_c = len(customers)
n_m = len(VEHICLES)
cust_idx = {c: i for i, c in enumerate(customers)}
Aeq = np.zeros((n_c, n_x))
for j, col in enumerate(columns):
    for c in col.served:
        Aeq[cust_idx[c], j] = 1.0
Aub = np.zeros((n_m, n_x))
for j, col in enumerate(columns):
    Aub[col.veh_idx, j] = 1.0
constraints = [
    LinearConstraint(Aeq, np.ones(n_c), np.ones(n_c)),
    LinearConstraint(Aub, -np.inf * np.ones(n_m), np.array([v.count_limit for v in VEHICLES], dtype=float)),
]
c = np.array([col.cost for col in columns], dtype=float)
integrality = np.ones(n_x, dtype=int)
bounds = Bounds(np.zeros(n_x), np.ones(n_x))
res = milp(c=c, constraints=constraints, integrality=integrality, bounds=bounds)
if res.success and res.x is not None:
    chosen = [i for i, val in enumerate(res.x) if val > 0.5]
    if chosen:
        return chosen
except Exception:
    pass
uncovered = set(customers)
used_by_vehicle = {veh.idx: 0 for veh in VEHICLES}
chosen: List[int] = []
while uncovered:
    best_idx = None
    best_score = -1e30
    for i, col in enumerate(columns):
        if used_by_vehicle[col.veh_idx] >= VEHICLES[col.veh_idx].count_limit:
            continue
        gain_set = col.served & uncovered
        if not gain_set:
            continue
        score = len(gain_set) / max(col.cost, 1.0)
        if score > best_score:
            best_score = score
            best_idx = i
    if best_idx is None:
        raise RuntimeError("Integer recovery failed under hard vehicle constraints.")
    chosen.append(best_idx)
    used_by_vehicle[columns[best_idx].veh_idx] += 1
    uncovered -= columns[best_idx].served
return chosen
def main() -> None:
    base = r"c:\Users\HP\Desktop\有用"
    order_df = pd.read_excel(base + r"\q1_preprocessed_订单信息_形式客户.xlsx")
    tw_df = pd.read_excel(base + r"\q1_preprocessed_时间窗_形式客户.xlsx")
    dist_df = pd.read_excel(base + r"\q1_preprocessed_距离矩阵_形式客户.xlsx")
    map_df = pd.read_excel(base + r"\q1_preprocessed_映射表.xlsx")
    w = {int(r["客户编号(新)"]): float(r["总重量"]) for _, r in order_df.iterrows()}
    v = {int(r["客户编号(新)"]): float(r["总体积"]) for _, r in order_df.iterrows()}
    form_id = {int(r["客户编号(新)"]): str(r["形式客户ID"]) for _, r in order_df.iterrows()}
    orig_id = {int(r["客户编号(新)"]): int(r["原客户编号"]) for _, r in order_df.iterrows()}
    tw = {
        int(r["客户编号(新)"]): (
            parse_hhmm_to_hour(r["开始时间"]),
            parse_hhmm_to_hour(r["结束时间"]),
        )
        for _, r in tw_df.iterrows()
    }
    row_nodes = dist_df.iloc[:, 0].astype(int).tolist()
    col_nodes = [int(x) for x in dist_df.columns[1:]]
    mat = dist_df.iloc[:, 1:].to_numpy(float)
    ridx = {n: i for i, n in enumerate(row_nodes)}
    cidx = {n: i for i, n in enumerate(col_nodes)}
    nmax = max(max(row_nodes), max(col_nodes))
    dist = [[0.0 for _ in range(nmax + 1)] for _ in range(nmax + 1)]
    for i in row_nodes:
        for j in col_nodes:
            dist[i][j] = float(mat[ridx[i], cidx[j]])
    customers = sorted(w.keys())
    print(f"[INIT] start building columns for customers={len(customers)}", flush=True)
    columns = build_initial_columns(customers, w, v, tw, dist)
    if not columns:
        raise RuntimeError("No initial feasible columns.")
    print(f"[INIT] customers={len(customers)}, initial_columns={len(columns)}", flush=True)
    max_iter = MAX_ITER
    for _it in range(max_iter):
        iter_id = _it + 1
        print(f"[CG] iter={iter_id} start, column_pool={len(columns)}", flush=True)
        rmp, pi_vec, sigma_vec = solve_rmp(columns, customers)
        print(f"[CG] iter={iter_id} rmp_obj={rmp.fun:.4f}", flush=True)

```

```

pi = {c: float(pi_vec[idx]) for idx, c in enumerate(customers)}
new_cols: List[Column] = []
for veh in VEHICLES:
    cols_m = pricing_dynamic(
        veh,
        customers,
        pi,
        float(sigma_vec[veh.idx]),
        w,
        v,
        tw,
        dist,
    )
    print(
        f"[CG] iter={iter_id} vehicle={veh.name} priced_cols={len(cols_m)}",
        flush=True,
    )
    new_cols.extend(cols_m)
added = 0
existing_keys = {(c.veh_idx, tuple(c.route)) for c in columns}
for col in new_cols:
    key = (col.veh_idx, tuple(col.route))
    if key in existing_keys:
        continue
    rc = col.cost - sum(pi[x] for x in col.route) - float(sigma_vec[col.veh_idx])
    if rc < -ADD_COL_RC_TOL:
        columns.append(col)
        existing_keys.add(key)
        added += 1
print(
    f"[CG] iter={iter_id} new_candidates={len(new_cols)} added={added} pool={len(columns)}",
    flush=True,
)
if added == 0:
    print(f"[CG] iter={iter_id} stop: no improving columns", flush=True)
    break
print("[FINAL] solving final RMP", flush=True)
rmp, _, _ = solve_rmp(columns, customers)
print("[FINAL] solving exact MILP recovery", flush=True)
chosen_idx = integer_recover(columns, customers)
chosen_cols = [columns[i] for i in chosen_idx]
print(f"[FINAL] MILP selected_routes={len(chosen_cols)}", flush=True)
route_rows = []
arrival_rows = []
usage = {veh.idx: 0 for veh in VEHICLES}
total = {"startup": 0.0, "energy": 0.0, "carbon": 0.0, "tw": 0.0, "sum": 0.0, "distance": 0.0}
for rid, col in enumerate(chosen_cols, start=1):
    veh = VEHICLES[col.veh_idx]
    usage[veh.idx] += 1
    ev = eval_route(col.route, veh, w, v, tw, dist)
    total["startup"] += ev["startup_cost"]
    total["energy"] += ev["energy_cost"]
    total["carbon"] += ev["carbon_cost"]
    total["tw"] += ev["tw_cost"]
    total["sum"] += ev["total_cost"]
    total["distance"] += ev["distance"]
    route_rows.append(
        {
            "route_id": rid,
            "vehicle": veh.name,
            "virtual_nodes": "->".join(["0"] + [str(x) for x in col.route] + ["0"]),
            "virtual_ids": "->".join(["0#0"] + [form_id[x] for x in col.route] + ["0#0"]),
            "num_nodes": len(col.route),
            "departure_hour": ev["departure_hour"],
            "cost": ev["total_cost"],
            "startup": ev["startup_cost"],
            "energy": ev["energy_cost"],
            "carbon_cost": ev["carbon_cost"],
            "tw_cost": ev["tw_cost"],
            "distance_km": ev["distance"],
        }
    )
)
t = float(ev["departure_hour"])
load_w = sum(w[c] for c in col.route)
load_v = sum(v[c] for c in col.route)
prev = 0
for c in col.route:
    d = dist[prev][c]
    tt, _, _ = leg_cost(veh, d, t, load_w)
    t += tt
    a, b = tw[c]
    before_w = load_w
    before_v = load_v
    after_w = before_w - w[c]
    after_v = before_v - v[c]

```



```

        arrival_rows.append(
            {
                "route_id": rid,
                "vehicle": veh.name,
                "virtual_customer": c,
                "virtual_id": form_id[c],
                "orig_customer": orig_id[c],
                "arrival_hour": round(t, 4),
                "tw_start_hour": a,
                "tw_end_hour": b,
                "early_h": max(a - t, 0.0),
                "late_h": max(t - b, 0.0),
                "load_weight_before_kg": round(before_w, 4),
                "deliver_weight_kg": round(w[c], 4),
                "load_weight_after_kg": round(after_w, 4),
                "load_volume_before_m3": round(before_v, 4),
                "deliver_volume_m3": round(v[c], 4),
                "load_volume_after_m3": round(after_v, 4),
            }
        )
        if t < a:
            t = a
        t += SERVICE_HOURS
        load_w = after_w
        load_v = after_v
        prev = c
    usage_rows = [
        {"vehicle": veh.name, "used": usage[veh.idx], "limit": veh.count_limit} for veh in VEHICLES
    ]
    pd.DataFrame(route_rows).to_excel(base + r"\q1_dw_solution_strict_routes.xlsx", index=False)
    pd.DataFrame(arrival_rows).to_excel(base + r"\q1_dw_solution_strict_arrivals.xlsx", index=False)
    pd.DataFrame(usage_rows).to_excel(base + r"\q1_dw_solution_strict_vehicle_usage.xlsx", index=False)
    map_df.to_excel(base + r"\q1_dw_solution_strict_virtual_mapping.xlsx", index=False)
    with open(base + r"\q1_dw_solution_strict_summary.txt", "w", encoding="utf-8") as f:
        f.write("Q1 strict Dantzig-Wolfe solution (exact pricing + exact MILP recovery)\n")
        f.write(f"Columns in pool: {len(columns)}\n")
        f.write(f"RMP LP objective: {rmp.fun:.2f}\n")
        f.write(f"Integer recovered routes: {len(chosen_cols)}\n")
        f.write("Vehicle usage:\n")
        for row in usage_rows:
            f.write(f" {row['vehicle']}: {row['used']}/{row['limit']}\n")
        f.write("\nRecovered integer cost breakdown (RMB):\n")
        f.write(f" Startup: {total['startup']:.2f}\n")
        f.write(f" Energy: {total['energy']:.2f}\n")
        f.write(f" Carbon: {total['carbon']:.2f}\n")
        f.write(f" TimeWindow: {total['tw']:.2f}\n")
        f.write(f" TOTAL: {total['sum']:.2f}\n")
        f.write(f" Distance(km): {total['distance']:.2f}\n")
    print("Done.")
    print(base + r"\q1_dw_solution_strict_summary.txt")
if __name__ == "__main__":
    main()
import pathlib
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np
plt.rcParams["font.sans-serif"] = ["Microsoft YaHei", "SimHei", "Arial Unicode MS", "DejaVu Sans"]
plt.rcParams["axes.unicode_minus"] = False
def parse_route_nodes(route_nodes: str):
    parts = str(route_nodes).split(">")
    return [int(x) for x in parts if str(x).strip() != ""]
def read_first_existing(base: pathlib.Path, candidates: list[str]) -> pd.DataFrame:
    for name in candidates:
        p = base / name
        if p.exists():
            return pd.read_excel(p)
    raise FileNotFoundError(f"None of candidate files exist: {candidates}")
def pick_col(df: pd.DataFrame, name_candidates: list[str], fallback_idx: int):
    for c in name_candidates:
        if c in df.columns:
            return c
    if 0 <= fallback_idx < len(df.columns):
        return df.columns[fallback_idx]
    raise KeyError(f"Cannot find expected column in DataFrame: {name_candidates}")
def main():
    base = pathlib.Path(r"c:\Users\HP\Desktop")
    coords_df = read_first_existing(
        base,
        [
            "q1_preprocessed_客户坐标_形式客户.xlsx",
            "temp_extract_客户坐标信息.xlsx",
            "temp_extract_客户坐标信息.xlsx",
        ],
    )
    routes_df = pd.read_excel(base / "q1_dw_solution_routes.xlsx")

```

```

coord_map = {
    int(r["ID"]): (float(r["X (km)"]), float(r["Y (km)"]))
    for _, r in coords_df.iterrows()
}
depot_xy = coord_map[0]

color_map = {
    "F3000": "#d62728",
    "F1500": "#ff7f0e",
    "F1250": "#9467bd",
    "E3000": "#1f77b4",
    "E1250": "#2ca02c",
}

x_min = float(coords_df["X (km)"].min()) - 3.0
x_max = float(coords_df["X (km)"].max()) + 3.0
y_min = float(coords_df["Y (km)"].min()) - 3.0
y_max = float(coords_df["Y (km)"].max()) + 3.0
all_vehicle_types = sorted(routes_df["vehicle"].astype(str).unique().tolist())
mapping_df = read_first_existing(
    base,
    [
        "q1_preprocessed_映射表.xlsx",
        "q1_dw_solution_virtual_mapping.xlsx",
        "q1p_solution_virtual_mapping.xlsx",
    ],
)
virt_to_orig = {}
virt_to_idx = {}
virt_to_cnt = {}
virt_to_w = {}
map_col_vid = pick_col(mapping_df, ["形式客户 ID"], 1)
map_col_orig = pick_col(mapping_df, ["原客户编号"], 2)
map_col_idx = pick_col(mapping_df, ["第几份"], 3)
map_col_cnt = pick_col(mapping_df, ["拆份数"], 4)
map_col_w = pick_col(mapping_df, ["拆分重量"], 5)
map_col_newid = pick_col(mapping_df, ["新客户编号"], 0)
for _, r in mapping_df.iterrows():
    vid = str(r[map_col_vid])
    virt_to_orig[vid] = int(r[map_col_orig])
    virt_to_idx[vid] = int(r[map_col_idx])
    virt_to_cnt[vid] = int(r[map_col_cnt])
    virt_to_w[vid] = float(r[map_col_w])
orig_xy = {}
coord_col_orig = "原客户 ID" if "原客户 ID" in coords_df.columns else "ID"
for _, r in coords_df.iterrows():
    oid = int(r[coord_col_orig]) if str(r[coord_col_orig]).isdigit() else None
    if oid is not None and oid != 0:
        orig_xy[oid] = (float(r["X (km)"]), float(r["Y (km)"]))
split_xy = {}
for _, r in mapping_df.iterrows():
    vid = str(r[map_col_vid])
    oid = int(r[map_col_orig])
    if oid not in orig_xy:
        continue
    x0, y0 = orig_xy[oid]
    idx = int(r[map_col_idx])
    cnt = int(r[map_col_cnt])

    radius = 0.18 + 0.03 * min(cnt, 10)
    angle = 2.0 * np.pi * (idx - 1) / max(cnt, 1)
    split_xy[vid] = (x0 + radius * np.cos(angle), y0 + radius * np.sin(angle))
id_to_vid = {}
for _, r in mapping_df.iterrows():
    id_to_vid[int(r[map_col_newid])] = str(r[map_col_vid])
def draw_plot(df: pd.DataFrame, title: str, out_file: pathlib.Path, single_vehicle: str | None = None):
    fig, ax = plt.subplots(figsize=(11.5, 9.0), dpi=180)
    fig.patch.set_facecolor("white")
    ax.set_facecolor("#fbfbfd")
    cust_df = coords_df[coords_df["ID"] != 0]
    ax.scatter(
        cust_df["X (km)"],
        cust_df["Y (km)"],
        s=14,
        facecolors="none",
        edgecolors="#b8bec7",
        linewidths=0.8,
        alpha=0.7,
        label="原始客户位置",
        zorder=1,
    )

    for _, row in df.iterrows():
        veh = str(row["vehicle"])
        nodes = parse_route_nodes(row["virtual_nodes"])

```

```

pts = []
for n in nodes:
    if n == 0:
        pts.append(coord_map[0])
    else:
        vid = id_to_vid.get(n)
        if vid in split_xy:
            pts.append(split_xy[vid])
        elif n in coord_map:
            pts.append(coord_map[n])
if len(pts) < 2:
    continue
xs = [p[0] for p in pts]
ys = [p[1] for p in pts]
ax.plot(
    xs,
    ys,
    color=color_map.get(veh, "#7f7f7f"),
    linewidth=1.5 if single_vehicle else 0.9,
    alpha=0.72 if single_vehicle else 0.22,
    zorder=2,
)
split_pts = []
split_sizes = []
for vid, (x, y) in split_xy.items():
    split_pts.append((x, y))
    split_sizes.append(8 + 0.02 * virt_to_w.get(vid, 0.0))
if split_pts:
    sx = [p[0] for p in split_pts]
    sy = [p[1] for p in split_pts]
    ax.scatter(
        sx,
        sy,
        s=split_sizes,
        c="#5b8ff9",
        alpha=0.55 if single_vehicle else 0.42,
        marker="o",
        linewidths=0.4,
        edgecolors="#ffffff",
        label="拆分后的形式客户点",
        zorder=3,
    )

ax.scatter(
    [depot_xy[0]],
    [depot_xy[1]],
    s=220,
    c="black",
    marker="*",
    label="配送中心",
    zorder=5,
)

ax.set_title(title, fontsize=15, fontweight="bold", pad=12)
ax.set_xlabel("X 坐标 (km)", fontsize=11)
ax.set_ylabel("Y 坐标 (km)", fontsize=11)
ax.set_xlim(x_min, x_max)
ax.set_ylim(y_min, y_max)
ax.grid(color="#d8dbe2", alpha=0.35, linewidth=0.8)
ax.set_aspect("equal", adjustable="box")
if single_vehicle is None:
    for veh, c in color_map.items():
        ax.plot([], [], color=c, linewidth=2.2, label=veh)
    ax.legend(loc="upper right", fontsize=9, ncol=2, frameon=True, framealpha=0.92)
else:
    route_count = len(df)
    ax.plot([], [], color=color_map.get(single_vehicle, "#7f7f7f"), linewidth=2.4, label=f"{single_vehicle} 路线数: {route_count}")
    ax.legend(loc="upper right", fontsize=9, frameon=True, framealpha=0.92)
fig.tight_layout()
fig.savefig(out_file)
plt.close(fig)
print(str(out_file))
draw_plot(
    routes_df,
    "Q1 路线图 (全部车型)",
    base / "q1_dw_route_map.png",
    single_vehicle=None,
)
for veh in all_vehicle_types:
    sub = routes_df[routes_df["vehicle"].astype(str) == veh].copy()
    if sub.empty:
        continue
    draw_plot(
        sub,
        f"Q1 路线图 - {veh}",

```

```

        base / f"q1_dw_route_map_{veh}.png",
        single_vehicle=veh,
    )
    clean = routes_df.copy()
    if "distance_km" in clean.columns:
        clean = clean.sort_values("distance_km", ascending=False).head(min(80, len(clean)))
    draw_plot(
        clean,
        "Q1 路线图（简洁视图：形式客户点 + 主要路线）",
        base / "q1_dw_route_map_clean.png",
        single_vehicle=None,
    )
if __name__ == "__main__":
    main()

```

ALSN 算法求解

```

import argparse
import math
import random
from dataclasses import dataclass, field
from pathlib import Path
from typing import Dict, List, Tuple
import numpy as np
import pandas as pd
@dataclass
class VehicleType:
    vid: int
    name: str
    weight_cap: float
    vol_cap: float
    count_cap: int
    is_electric: bool
@dataclass
class Task:
    tid: int
    cust_id: int
    weight: float
    volume: float
    tw_start: float
    tw_end: float
@dataclass
class Route:
    vehicle: VehicleType
    task_ids: List[int] = field(default_factory=list)
def speed_kmh(t_min: float) -> float:
    if 0 <= t_min < 60:
        return 9.8
    if 60 <= t_min < 120:
        return 55.3
    if 120 <= t_min < 210:
        return 35.4
    if 210 <= t_min < 300:
        return 9.8
    if 300 <= t_min < 420:
        return 55.3
    if 420 <= t_min < 540:
        return 35.4
    return 35.4
def segment_end(t_min: float) -> float:
    cuts = [60, 120, 210, 300, 420, 540]
    for c in cuts:
        if t_min < c:
            return t_min + 60
def travel_energy(distance_km: float, depart_min: float, is_electric: bool, load_ratio: float) -> Tuple[float, float]:
    rem = distance_km
    t = depart_min
    travel_min = 0.0
    energy = 0.0
    while rem > 1e-9:
        v = speed_kmh(t)
        end = segment_end(t)
        dt_h = max((end - t) / 60.0, 1e-9)
        can_run = v * dt_h
        run = min(can_run, rem)
        use_h = run / v
        rem -= run
        t += use_h * 60.0
        travel_min += use_h * 60.0
        if is_electric:
            epk = 0.001 * v * v - 0.1 * v + 36.19
            epk *= 1.0 + 0.35 * load_ratio
            energy += run / 100.0 * epk
    else:

```

```

        fpk = 0.0025 * v * v - 0.2554 * v + 31.75
        fpk *= 1.0 + 0.40 * load_ratio
        energy += run / 100.0 * fpk
    return travel_min, energy
class ALNSSolver:
    def __init__(self, base_dir: Path, customer_min: int = 1, customer_max: int = 98, seed: int = 42):
        self.base = base_dir
        self.customer_min = customer_min
        self.customer_max = customer_max
        self.rng = random.Random(seed)
        self.service_min = 20.0
        self.start_cost = 400.0
        self.early_cost = 20.0 / 60.0
        self.late_cost = 50.0 / 60.0
        self.fuel_price = 7.61
        self.elec_price = 1.64
        self.carbon_price = 0.65
        self.fuel_cf = 2.547
        self.elec_cf = 0.501
        self.vehicle_types = [
            VehicleType(1, "Fuel-3000", 3000.0, 13.5, 60, False),
            VehicleType(2, "Fuel-1500", 1500.0, 10.8, 50, False),
            VehicleType(3, "Fuel-1250", 1250.0, 6.5, 50, False),
            VehicleType(4, "EV-3000", 3000.0, 15.0, 10, True),
            VehicleType(5, "EV-1250", 1250.0, 8.5, 15, True),
        ]
        self.max_w = max(v.weight_cap for v in self.vehicle_types)
        self.max_v = max(v.vol_cap for v in self.vehicle_types)
    def load_data(self):
        coord = pd.read_excel(self.base / "temp_extract_客户坐标信息.xlsx")
        coord.columns = ["node_type", "id", "x", "y"]
        tw = pd.read_excel(self.base / "temp_extract_时间窗.xlsx")
        tw.columns = ["id", "start", "end"]
        od = pd.read_excel(self.base / "temp_extract_订单信息.xlsx")
        od.columns = ["order_id", "weight", "volume", "cust_id"]
        dm = pd.read_excel(self.base / "temp_extract_距离矩阵.xlsx")
        od["weight"] = pd.to_numeric(od["weight"], errors="coerce").fillna(0.0)
        od["volume"] = pd.to_numeric(od["volume"], errors="coerce").fillna(0.0)
        od["cust_id"] = pd.to_numeric(od["cust_id"], errors="coerce").fillna(0).astype(int)
        od = od[(od["cust_id"] >= self.customer_min) & (od["cust_id"] <= self.customer_max)]
        self.ids = sorted(coord["id"].astype(int).tolist())
        self.id_pos = {nid: i for i, nid in enumerate(self.ids)}
        self.dist_mat = dm.iloc[:, 1:].to_numpy(float)
        tws, twe = {}, {}
        for _, r in tw.iterrows():
            cid = int(r["id"])
            s = pd.to_datetime(str(r["start"]))
            e = pd.to_datetime(str(r["end"]))
            tws[cid] = float((s.hour * 60 + s.minute) - 8 * 60)
            twe[cid] = float((e.hour * 60 + e.minute) - 8 * 60)
        agg = od.groupby("cust_id", as_index=False)[["weight", "volume"]].sum()
        tasks = []
        split_orders = 0
        for _, r in agg.iterrows():
            cid = int(r["cust_id"])
            if cid not in tws:
                continue
            w, v = float(r["weight"]), float(r["volume"])
            if w <= 1e-9 and v <= 1e-9:
                continue
            parts = max(1, int(math.ceil(max(w / self.max_w, v / self.max_v))))
            if parts > 1:
                split_orders += 1
            for p in range(parts):
                tid = cid * 100 + p
                tasks.append(Task(tid, cid, w / parts, v / parts, tws[cid], twe[cid]))
        tasks.sort(key=lambda t: (t.tw_end, t.tw_start))
        self.tasks = tasks
        self.task_map = {t.tid: t for t in tasks}
        self.split_orders = split_orders
    def dist(self, a: int, b: int) -> float:
        return float(self.dist_mat[self.id_pos[a], self.id_pos[b]])
    def route_cost(self, r: Route, depart_min: float = 0.0) -> Tuple[float, float, float, float, float]:
        if not r.task_ids:
            return 0.0, 0.0, 0.0, 0.0, depart_min
        t = depart_min
        ec = 0.0
        cc = 0.0
        twc = 0.0
        total_w = sum(self.task_map[tid].weight for tid in r.task_ids)
        cur_w = total_w
        prev = 0
        for tid in r.task_ids:
            task = self.task_map[tid]
            lr = min(max(cur_w / r.vehicle.weight_cap, 0.0), 1.0)

```

```

dt, en = travel_energy(self.dist(prev, task.cust_id), t, r.vehicle.is_electric, lr)
t += dt
if r.vehicle.is_electric:
    ec += en * self.elec_price
    cc += en * self.elec_cf * self.carbon_price
else:
    ec += en * self.fuel_price
    cc += en * self.fuel_cf * self.carbon_price
wait = max(0.0, task.tw_start - t)
t += wait
late = max(0.0, t - task.tw_end)
twc += self.early_cost * wait + self.late_cost * late
t += self.service_min
cur_w += task.weight
prev = task.cust_id
dt2, en2 = travel_energy(self.dist(prev, 0), t, r.vehicle.is_electric, 0.0)
if r.vehicle.is_electric:
    ec += en2 * self.elec_price
    cc += en2 * self.elec_cf * self.carbon_price
else:
    ec += en2 * self.fuel_price
    cc += en2 * self.fuel_cf * self.carbon_price
return self.start_cost + ec + cc + twc, ec, cc, twc, t
def feasible(self, r: Route) -> bool:
    w = sum(self.task_map[tid].weight for tid in r.task_ids)
    v = sum(self.task_map[tid].volume for tid in r.task_ids)
    return w <= r.vehicle.weight_cap + 1e-9 and v <= r.vehicle.vol_cap + 1e-9
def evaluate_solution(self, routes: List[Route]) -> Dict[str, object]:
    turnaround_min = 15.0
    trips_by_type: Dict[int, List[Route]] = {v.vid: [] for v in self.vehicle_types}
    for r in routes:
        if r.task_ids:
            trips_by_type[r.vehicle.vid].append(r)
    total = 0.0
    energy = 0.0
    carbon = 0.0
    twc = 0.0
    trip_count = {v.vid: 0 for v in self.vehicle_types}
    peak_used = {v.vid: 0 for v in self.vehicle_types}
    for v in self.vehicle_types:
        rlist = trips_by_type[v.vid]
        if not rlist:
            continue
        rlist = sorted(rlist, key=lambda rr: min(self.task_map[tid].tw_end for tid in rr.task_ids))
        avail = [0.0 for _ in range(v.count_cap)]
        for rr in rlist:
            k = int(np.argmax(avail))
            depart = avail[k]
            c, ec, cc, tw, finish = self.route_cost(rr, depart_min=depart)
            avail[k] = finish + turnaround_min
            total += c
            energy += ec
            carbon += cc
            twc += tw
            trip_count[v.vid] += 1
        peak_used[v.vid] = sum(1 for t in avail if t > 0.0)
    return {
        "total_cost": total,
        "energy_cost": energy,
        "carbon_cost": carbon,
        "time_window_cost": twc,
        "num_routes": float(sum(trip_count.values())),
        "trip_count": trip_count,
        "peak_used": peak_used,
    }
def total_cost(self, routes: List[Route]) -> float:
    return float(self.evaluate_solution(routes)["total_cost"])
def initial_solution(self) -> List[Route]:
    routes: List[Route] = []
    for task in self.tasks:
        best = None
        best_delta = 1e18
        for i, r in enumerate(routes):
            base = self.route_cost(r)[0]
            trial = Route(r.vehicle, r.task_ids + [task.tid])
            if not self.feasible(trial):
                continue
            new = self.route_cost(trial)[0]
            d = new - base
            if d < best_delta:
                best_delta = d
                best = ("append", i)
        for v in self.vehicle_types:
            trial = Route(v, [task.tid])
            if not self.feasible(trial):

```

```

        continue
    d = self.route_cost(trial)[0]
    if d < best_delta:
        best_delta = d
        best = ("new", v)
    if best[0] == "append":
        routes[best[1]].task_ids.append(task.tid)
    else:
        routes.append(Route(best[1], [task.tid]))
    return [r for r in routes if r.task_ids]
def destroy(self, routes: List[Route], remove_n: int) -> Tuple[List[Route], List[int]]:
    all_tasks = [(ri, ti) for ri, r in enumerate(routes) for ti in range(len(r.task_ids))]
    self.rng.shuffle(all_tasks)
    removed_ids = []
    for ri, ti in all_tasks[:remove_n]:
        if ti < len(routes[ri].task_ids):
            removed_ids.append(routes[ri].task_ids[ti])
            routes[ri].task_ids.pop(ti)
    routes = [r for r in routes if r.task_ids]
    return routes, removed_ids
def repair(self, routes: List[Route], removed: List[int]) -> List[Route]:
    self.rng.shuffle(removed)
    for tid in removed:
        best = None
        best_delta = 1e18
        for i, r in enumerate(routes):
            base = self.route_cost(r)[0]
            for pos in range(len(r.task_ids) + 1):
                cand_ids = r.task_ids[:pos] + [tid] + r.task_ids[pos:]
                trial = Route(r.vehicle, cand_ids)
                if not self.feasible(trial):
                    continue
                new = self.route_cost(trial)[0]
                d = new - base
                if d < best_delta:
                    best_delta = d
                    best = ("insert", i, pos)
        for v in self.vehicle_types:
            trial = Route(v, [tid])
            if not self.feasible(trial):
                continue
            d = self.route_cost(trial)[0]
            if d < best_delta:
                best_delta = d
                best = ("new", v)
        if best[0] == "insert":
            routes[best[1]].task_ids.insert(best[2], tid)
        else:
            routes.append(Route(best[1], [tid]))
    return routes
def local_search(self, routes: List[Route]) -> List[Route]:
    improved = True
    while improved:
        improved = False
        base_total = self.total_cost(routes)
        for i in range(len(routes)):
            for p in range(len(routes[i].task_ids)):
                tid = routes[i].task_ids[p]
                for j in range(len(routes)):
                    if i == j:
                        continue
                    for q in range(len(routes[j].task_ids) + 1):
                        ra = Route(routes[i].vehicle, routes[i].task_ids[:p] + routes[i].task_ids[p + 1 :])
                        rb = Route(routes[j].vehicle, routes[j].task_ids[:q] + [tid] + routes[j].task_ids[q:])
                        if (not ra.task_ids) or (not self.feasible(rb)):
                            continue
                        cand = routes.copy()
                        cand[i], cand[j] = ra, rb
                        cand = [r for r in cand if r.task_ids]
                        c = self.total_cost(cand)
                        if c + 1e-6 < base_total:
                            routes = cand
                            improved = True
                            break
                    if improved:
                        break
                if improved:
                    break
            if improved:
                break
        for i, r in enumerate(routes):
            best = r.task_ids
            best_c = self.route_cost(r)[0]

```

```

        n = len(r.task_ids)
        for a in range(n):
            for b in range(a + 1, n):
                cand = r.task_ids[:a] + list(reversed(r.task_ids[a : b + 1])) + r.task_ids[b + 1 :]
                trial = Route(r.vehicle, cand)
                c = self.route_cost(trial)[0]
                if c < best_c:
                    best_c = c
                    best = cand
            routes[i].task_ids = best
        return routes
def vehicle_trip_counts(self, routes: List[Route]) -> Dict[int, int]:
    out = {v.vid: 0 for v in self.vehicle_types}
    for r in routes:
        out[r.vehicle.vid] += 1
    return out
def solve(self, iterations: int = 800) -> Tuple[List[Route], Dict[str, float]]:
    self.load_data()
    cur = self.initial_solution()
    cur_cost = self.total_cost(cur)
    print(f"[ALNS] init_solution cost={cur_cost:.2f}, routes={len(cur)}, tasks={len(self.tasks)}")
    best = [Route(r.vehicle, list(r.task_ids)) for r in cur]
    best_cost = cur_cost
    temp = 500.0
    for it in range(1, iterations + 1):
        cand = [Route(r.vehicle, list(r.task_ids)) for r in cur]
        remove_n = max(3, int(0.08 * len(self.tasks)))
        cand, removed = self.destroy(cand, remove_n)
        cand = self.repair(cand, removed)
        if it % 20 == 0:
            cand = self.local_search(cand)
        cand_cost = self.total_cost(cand)
        accept = cand_cost < cur_cost
        if not accept:
            prob = math.exp(min(0.0, (cur_cost - cand_cost) / max(temp, 1e-6)))
            accept = self.rng.random() < prob
        if accept:
            cur, cur_cost = cand, cand_cost
        if cand_cost < best_cost:
            best, best_cost = cand, cand_cost
        temp *= 0.997
        if it % 20 == 0:
            print(f"[ALNS] iter={it}, current={cur_cost:.2f}, best={best_cost:.2f}, routes={len(best)}")
    eval_best = self.evaluate_solution(best)
    vc = eval_best["trip_count"]
    pv = eval_best["peak_used"]
    summary = {
        "total_cost": float(eval_best["total_cost"]),
        "num_routes": float(eval_best["num_routes"]),
        "start_cost": float(eval_best["num_routes"] * self.start_cost),
        "energy_cost": float(eval_best["energy_cost"]),
        "carbon_cost": float(eval_best["carbon_cost"]),
        "time_window_cost": float(eval_best["time_window_cost"]),
        "num_tasks": float(len(self.tasks)),
        "num_split_orders": float(self.split_orders),
    }
    for v in self.vehicle_types:
        summary[f"vehicle_type_{v.vid}_trips"] = float(vc[v.vid])
        summary[f"vehicle_type_{v.vid}_peak_vehicles"] = float(pv[v.vid])
    return best, summary
def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--data-dir", type=str, default=r"d:\Users\HP\xwechat_files\wxid_wnqdm1u4y5rf22_ff5f\msg\file\2026-04")
    parser.add_argument("--customer-min", type=int, default=1)
    parser.add_argument("--customer-max", type=int, default=98)
    parser.add_argument("--iters", type=int, default=800)
    parser.add_argument("--seed", type=int, default=42)
    parser.add_argument("--save", action="store_true")
    args = parser.parse_args()
    solver = ALNSolver(Path(args.data_dir), args.customer_min, args.customer_max, seed=args.seed)
    routes, summary = solver.solve(iterations=args.iters)
    print("\n=== ALNS Q1 SUMMARY ===")
    for k, v in summary.items():
        if isinstance(v, float):
            print(f"{k}: {v:.6f}")
        else:
            print(f"{k}: {v}")
    if args.save:
        out = Path(r"d:\temp\q1_alns")
        out.mkdir(parents=True, exist_ok=True)
        rows = []
        for i, r in enumerate(routes, 1):
            rows.append(
                {
                    "route_id": i,

```



```

        "vehicle_type": r.vehicle.vid,
        "vehicle_name": r.vehicle.name,
        "num_tasks": len(r.task_ids),
        "path_customers": "->".join(["0"] + [str(solver.task_map[tid].cust_id) for tid in r.task_ids] + ["0"]),
        "route_cost": solver.route_cost(r)[0],
    }
)
pd.DataFrame(rows).to_csv(out / "routes.csv", index=False, encoding="utf-8-sig")
pd.DataFrame([summary]).to_csv(out / "summary.csv", index=False, encoding="utf-8-sig")
print(f"saved_to: {out}")
if __name__ == "__main__":
    main()

```

附录3 问题二

```

import math
from dataclasses import dataclass
from typing import Dict, List, Optional, Set, Tuple
import numpy as np
import pandas as pd
from scipy.optimize import linprog
SERVICE_HOURS = 20.0 / 60.0
STARTUP_COST = 400.0
EARLY_COST_PER_HOUR = 20.0
LATE_COST_PER_HOUR = 50.0
CARBON_COST_PER_KG = 0.65
DEPARTURE_CANDIDATES = [8.0, 10.0, 12.0, 14.0, 16.0, 18.0, 20.0]
BAN_END = 16.0
# Run mode tuning
PRE_RUN_MODE = True
if PRE_RUN_MODE:
    MAX_ITER = 30
    PRICING_RC_TOL = 1e-4
    ADD_COL_RC_TOL = 1e-4
else:
    MAX_ITER = 30
    PRICING_RC_TOL = 1e-6
    ADD_COL_RC_TOL = 1e-5
_ROUTE_BEST_CACHE: Dict[Tuple[int, Tuple[int, ...], Tuple[float, ...]], Dict[str, float]] = {}
_ROUTE_FIXED_CACHE: Dict[Tuple[int, Tuple[int, ...], float], Dict[str, float]] = {}
@dataclass(frozen=True)
class VehicleType:
    idx: int
    name: str
    kind: str
    capacity_weight: float
    capacity_volume: float
    count_limit: int
    energy_price: float
    carbon_coeff: float
    full_load_increase: float
VEHICLES: List[VehicleType] = [
    VehicleType(0, "F3000", "fuel", 3000.0, 13.5, 60, 7.61, 2.547, 0.40),
    VehicleType(1, "F1500", "fuel", 1500.0, 10.8, 50, 7.61, 2.547, 0.40),
    VehicleType(2, "F1250", "fuel", 1250.0, 6.5, 50, 7.61, 2.547, 0.40),
    VehicleType(3, "E3000", "ev", 3000.0, 15.0, 10, 1.64, 0.501, 0.35),
    VehicleType(4, "E1250", "ev", 1250.0, 8.5, 15, 1.64, 0.501, 0.35),
]
def parse_hhmm_to_hour(x: str) -> float:
    h, m = str(x).split(":")
    return int(h) + int(m) / 60.0
def segments() -> List[Tuple[float, float, float]]:
    return [
        (8.0, 9.0, 9.8),
        (9.0, 10.0, 55.3),
        (10.0, 11.5, 35.4),
        (11.5, 13.0, 9.8),
        (13.0, 15.0, 55.3),
        (15.0, 17.0, 35.4),
        (17.0, 19.0, 9.8),
        (19.0, 20.0, 55.3),
    ]
def speed_at(t: float) -> float:
    x = t % 24.0
    for s, e, v in segments():
        if s <= x < e:
            return v
    return 35.4
def next_boundary(t: float) -> float:
    x = t % 24.0
    for s, e, _ in segments():
        if s <= x < e:
            return t + (e - x)
    return math.floor(t) + 1.0

```

```

def travel_profile(distance_km: float, depart: float) -> Tuple[float, List[Tuple[float, float]]]:
    remain = distance_km
    t = depart
    elapsed = 0.0
    segs: List[Tuple[float, float]] = []
    while remain > 1e-9:
        v = speed_at(t)
        b = next_boundary(t)
        dt = max(1e-6, b - t)
        can_go = v * dt
        if can_go >= remain:
            segs.append((remain, v))
            elapsed += remain / v
            return elapsed, segs
        segs.append((can_go, v))
        remain -= can_go
        elapsed += dt
        t += dt
    return elapsed, segs

def energy_per_100km(v: float, kind: str) -> float:
    if kind == "fuel":
        return 0.0025 * v * v - 0.2554 * v + 31.75
    return 0.0014 * v * v - 0.12 * v + 36.19

def leg_cost(veh: VehicleType, d: float, depart: float, load_w: float) -> Tuple[float, float, float]:
    tt, segs = travel_profile(d, depart)
    e0 = 0.0
    for seg_d, seg_v in segs:
        e0 += energy_per_100km(seg_v, veh.kind) * seg_d / 100.0
    load_ratio = min(1.0, max(0.0, load_w / veh.capacity_weight))
    e = e0 * (1.0 + veh.full_load_increase * load_ratio)
    e_cost = e * veh.energy_price
    c_cost = e * veh.carbon_coeff * CARBON_COST_PER_KG
    return tt, e_cost, c_cost

def violates_green_policy(veh: VehicleType, customer: int, arrival_t: float, green_set: Set[int]) -> bool:
    return veh.kind == "fuel" and (customer in green_set) and (arrival_t < BAN_END)

def eval_route_at_start(
    route: List[int],
    veh: VehicleType,
    w: Dict[int, float],
    v: Dict[int, float],
    tw: Dict[int, Tuple[float, float]],
    dist: List[List[float]],
    start_t: float,
    green_set: Set[int],
) -> Dict[str, float]:
    sw = sum(w[c] for c in route)
    sv = sum(v[c] for c in route)
    if sw > veh.capacity_weight + 1e-9 or sv > veh.capacity_volume + 1e-9:
        return {"feasible": 0.0, "total_cost": 1e30}
    t = start_t
    load_w = sw
    startup = STARTUP_COST
    energy = 0.0
    carbon = 0.0
    tw_cost = 0.0
    dist_sum = 0.0
    prev = 0
    for c in route:
        d = dist[prev][c]
        dist_sum += d
        tt, ec, cc = leg_cost(veh, d, t, load_w)
        t += tt
        if violates_green_policy(veh, c, t, green_set):
            return {"feasible": 0.0, "total_cost": 1e30}
        energy += ec
        carbon += cc
        a, b = tw[c]
        early = max(a - t, 0.0)
        late = max(t - b, 0.0)
        tw_cost += EARLY_COST_PER_HOUR * early + LATE_COST_PER_HOUR * late
        if t < a:
            t = a
        t += SERVICE_HOURS
        load_w -= w[c]
        prev = c
    d_back = dist[prev][0]
    dist_sum += d_back
    tt, ec, cc = leg_cost(veh, d_back, t, load_w)
    _ = tt
    energy += ec
    carbon += cc
    total = startup + energy + carbon + tw_cost
    return {
        "feasible": 1.0,
        "total_cost": total,
    }

```

```

        "departure_hour": start_t,
        "startup_cost": startup,
        "energy_cost": energy,
        "carbon_cost": carbon,
        "tw_cost": tw_cost,
        "distance": dist_sum,
    }
def eval_route(
    route: List[int],
    veh: VehicleType,
    w: Dict[int, float],
    v: Dict[int, float],
    tw: Dict[int, Tuple[float, float]],
    dist: List[List[float]],
    green_set: Set[int],
    start_t: Optional[float] = None,
) -> Dict[str, float]:
    route_key = tuple(route)
    if start_t is not None:
        fixed_key = (veh.idx, route_key, float(start_t))
        cached = _ROUTE_FIXED_CACHE.get(fixed_key)
        if cached is not None:
            return cached
        ev = eval_route_at_start(route, veh, w, v, tw, dist, start_t, green_set)
        _ROUTE_FIXED_CACHE[fixed_key] = ev
        return ev
    best_key = (veh.idx, route_key, tuple(DEPARTURE_CANDIDATES))
    cached_best = _ROUTE_BEST_CACHE.get(best_key)
    if cached_best is not None:
        return cached_best
    best: Optional[Dict[str, float]] = None
    for dep in DEPARTURE_CANDIDATES:
        ev = eval_route_at_start(route, veh, w, v, tw, dist, dep, green_set)
        if ev["feasible"] < 0.5:
            continue
        if best is None or ev["total_cost"] < best["total_cost"]:
            best = ev
    if best is None:
        return {"feasible": 0.0, "total_cost": 1e30}
    _ROUTE_BEST_CACHE[best_key] = best
    return best

@dataclass
class Column:
    veh_idx: int
    route: List[int]
    served: Set[int]
    cost: float

def build_initial_columns(customers: List[int], w: Dict[int, float], v: Dict[int, float], tw, dist, green_set: Set[int]) -> List[Column]:
    cols: List[Column] = []
    order = sorted(
        customers,
        key=lambda c: (max(w[c] / 3000.0, v[c] / 15.0), w[c], v[c]),
        reverse=True,
    )
    used = {veh.idx: 0 for veh in VEHICLES}
    routes: List[Dict[str, object]] = []
    for c in order:
        bw = w[c]
        bv = v[c]
        best_r = None
        best_s = None
        for rid, r in enumerate(routes):
            veh = VEHICLES[int(r["veh_idx"])]
            rem_w = float(r["rem_w"])
            rem_v = float(r["rem_v"])
            if rem_w + 1e-9 < bw or rem_v + 1e-9 < bv:
                continue
            s = (rem_w - bw) / veh.capacity_weight + (rem_v - bv) / veh.capacity_volume
            if best_s is None or s < best_s:
                best_s = s
                best_r = rid
        if best_r is not None:
            routes[best_r]["nodes"].append(c)
            routes[best_r]["rem_w"] -= bw
            routes[best_r]["rem_v"] -= bv
            continue
        best_veh = None
        best_score = None
        for veh in VEHICLES:
            if used[veh.idx] >= veh.count_limit:
                continue
            if veh.capacity_weight + 1e-9 < bw or veh.capacity_volume + 1e-9 < bv:
                continue
            s = (veh.capacity_weight - bw) / veh.capacity_weight + (veh.capacity_volume - bv) / veh.capacity_volume
            if best_score is None or s < best_score:

```

```

        best_score = s
        best_veh = veh
    if best_veh is None:
        raise RuntimeError(f"Cannot build initial feasible routes for customer {c}")
    routes.append(
        {
            "veh_idx": best_veh.idx,
            "nodes": [c],
            "rem_w": best_veh.capacity_weight - bw,
            "rem_v": best_veh.capacity_volume - bv,
        }
    )
    used[best_veh.idx] += 1
for r in routes:
    nodes = list(r["nodes"])
    seq = []
    cur = 0
    while nodes:
        nxt = min(nodes, key=lambda n: (dist[cur][n], tw[n][0], n))
        seq.append(nxt)
        nodes.remove(nxt)
        cur = nxt
    veh = VEHICLES[int(r["veh_idx"])]
    ev = eval_route(seq, veh, w, v, tw, dist, green_set)
    cols.append(Column(veh.idx, seq, set(seq), float(ev["total_cost"])))
for c in customers:
    for veh in VEHICLES:
        ev = eval_route([c], veh, w, v, tw, dist, green_set)
        if ev["feasible"] > 0.5:
            cols.append(Column(veh.idx, [c], {c}, float(ev["total_cost"])))
print(f"[INIT] total initial columns: {len(cols)}", flush=True)
return cols
def solve_rmp(columns: List[Column], customers: List[int]):
    n_c = len(customers)
    n_m = len(VEHICLES)
    n_x = len(columns)
    cust_idx = {c: i for i, c in enumerate(customers)}
    A_eq = np.zeros((n_c, n_x))
    b_eq = np.ones(n_c)
    for j, col in enumerate(columns):
        for c in col.served:
            A_eq[cust_idx[c], j] = 1.0
    A_ub = np.zeros((n_m, n_x))
    b_ub = np.array([veh.count_limit for veh in VEHICLES], dtype=float)
    for j, col in enumerate(columns):
        A_ub[col.veh_idx, j] = 1.0
    c = np.array([col.cost for col in columns], dtype=float)
    bounds = [(0.0, None)] * n_x
    res = linprog(c, A_ub=A_ub, b_ub=b_ub, A_eq=A_eq, b_eq=b_eq, bounds=bounds, method="highs")
    if not res.success:
        raise RuntimeError(f"RMP failed: {res.message}")
    pi = np.array(res.eqlin.marginals, dtype=float)
    sigma = np.array(res.ineqlin.marginals, dtype=float)
    return res, pi, sigma
def pricing_dynamic(
    veh: VehicleType,
    customers: List[int],
    pi: Dict[int, float],
    sigma_m: float,
    w: Dict[int, float],
    v: Dict[int, float],
    tw: Dict[int, Tuple[float, float]],
    dist,
    green_set: Set[int],
    beam_width: int = 120,
    max_depth: int = 6,
) -> List[Column]:
    cand = sorted(customers, key=lambda c: pi[c], reverse=True)[:50]
    labels: List[Tuple[List[int], Set[int], float, float]] = []
    for c in cand:
        ev = eval_route([c], veh, w, v, tw, dist, green_set)
        if ev["feasible"] < 0.5:
            continue
        rc = ev["total_cost"] - pi[c] - sigma_m
        labels.append((c, {c}, rc, ev["total_cost"]))
    best_cols: List[Column] = []
    for _ in range(max_depth - 1):
        new_labels: List[Tuple[List[int], Set[int], float, float]] = []
        labels.sort(key=lambda x: x[2])
        labels = labels[:beam_width]
        for route, served, _, _ in labels:
            for c in cand:
                if c in served:
                    continue
                nr = route + [c]

```

```

        ev = eval_route(nr, veh, w, v, tw, dist, green_set)
        if ev["feasible"] < 0.5:
            continue
        dual_sum = sum(pi[x] for x in nr)
        rc = ev["total_cost"] - dual_sum - sigma_m
        new_labels.append((nr, set(nr), rc, ev["total_cost"]))
        if rc < -PRICING_RC_TOL:
            best_cols.append(Column(veh.idx, nr, set(nr), ev["total_cost"]))
    if not new_labels:
        break
    new_labels.sort(key=lambda x: x[2])
    labels = new_labels[: beam_width * 2]
for route, served, rc, rcost in labels:
    if rc < -PRICING_RC_TOL:
        best_cols.append(Column(veh.idx, route, served, rcost))
seen = set()
uniq: List[Column] = []
for col in sorted(best_cols, key=lambda x: x.cost):
    key = (col.veh_idx, tuple(col.route))
    if key in seen:
        continue
    seen.add(key)
    uniq.append(col)
    if len(uniq) >= 40:
        break
return uniq
def integer_recover(columns: List[Column], customers: List[int]) -> List[int]:
    try:
        from scipy.optimize import Bounds, LinearConstraint, milp
        n_x = len(columns)
        n_c = len(customers)
        n_m = len(VEHICLES)
        cust_idx = {c: i for i, c in enumerate(customers)}
        Aeq = np.zeros((n_c, n_x))
        for j, col in enumerate(columns):
            for c in col.served:
                Aeq[cust_idx[c], j] = 1.0
        Aub = np.zeros((n_m, n_x))
        for j, col in enumerate(columns):
            Aub[col.veh_idx, j] = 1.0
        constraints = [
            LinearConstraint(Aeq, np.ones(n_c), np.ones(n_c)),
            LinearConstraint(Aub, -np.inf * np.ones(n_m), np.array([v.count_limit for v in VEHICLES], dtype=float)),
        ]
        c = np.array([col.cost for col in columns], dtype=float)
        integrality = np.ones(n_x, dtype=int)
        bounds = Bounds(np.zeros(n_x), np.ones(n_x))
        res = milp(c=c, constraints=constraints, integrality=integrality, bounds=bounds)
        if res.success and res.x is not None:
            chosen = [i for i, val in enumerate(res.x) if val > 0.5]
            if chosen:
                return chosen
    except Exception:
        pass
    uncovered = set(customers)
    used_by_vehicle = {veh.idx: 0 for veh in VEHICLES}
    chosen: List[int] = []
    while uncovered:
        best_idx = None
        best_score = -1e30
        for i, col in enumerate(columns):
            if used_by_vehicle[col.veh_idx] >= VEHICLES[col.veh_idx].count_limit:
                continue
            gain_set = col.served & uncovered
            if not gain_set:
                continue
            score = len(gain_set) / max(col.cost, 1.0)
            if score > best_score:
                best_score = score
                best_idx = i
        if best_idx is None:
            raise RuntimeError("Integer recovery failed under hard vehicle constraints.")
        chosen.append(best_idx)
        used_by_vehicle[columns[best_idx].veh_idx] += 1
        uncovered -= columns[best_idx].served
    return chosen
def main() -> None:
    base = r"c:\Users\HP\Desktop\有用"
    order_df = pd.read_excel(base + r"\q1_preprocessed_订单信息_形式客户.xlsx")
    tw_df = pd.read_excel(base + r"\q1_preprocessed_时间窗_形式客户.xlsx")
    dist_df = pd.read_excel(base + r"\q1_preprocessed_距离矩阵_形式客户.xlsx")
    map_df = pd.read_excel(base + r"\q1_preprocessed_映射表.xlsx")
    coords_orig = pd.read_excel(base + r"\q1_preprocessed_客户坐标_形式客户.xlsx")
    w = {int(r["客户编号(新)"]): float(r["总重量"]) for _, r in order_df.iterrows()}
    v = {int(r["客户编号(新)"]): float(r["总体积"]) for _, r in order_df.iterrows()}

```

```

form_id = {int(r["客户编号(新)"]): str(r["形式客户 ID"]) for _, r in order_df.iterrows()}
orig_id = {int(r["客户编号(新)"]): int(r["原客户编号"]) for _, r in order_df.iterrows()}
coords_orig = coords_orig.iloc[:, :4].copy()
coords_orig.columns = ["类型", "ID", "X", "Y"]
c0 = coords_orig[coords_orig["类型"] != "配送中心"].copy()
c0["in_zone"] = c0["X"]**2 + c0["Y"]**2 <= 100.0 + 1e-9
green_orig = set(c0.loc[c0["in_zone"], "ID"].astype(int).tolist())
green_virtual = set(int(r["新客户编号"]) for _, r in map_df.iterrows() if int(r["原客户编号"]) in green_orig)
tw = {
    int(r["客户编号(新)"]): (
        parse_hhmm_to_hour(r["开始时间"]),
        parse_hhmm_to_hour(r["结束时间"]),
    )
    for _, r in tw_df.iterrows()
}
row_nodes = dist_df.iloc[:, 0].astype(int).tolist()
col_nodes = [int(x) for x in dist_df.columns[1:]]
mat = dist_df.iloc[:, 1:].to_numpy(float)
ridx = {n: i for i, n in enumerate(row_nodes)}
cidx = {n: i for i, n in enumerate(col_nodes)}
nmax = max(max(row_nodes), max(col_nodes))
dist = [[0.0 for _ in range(nmax + 1)] for _ in range(nmax + 1)]
for i in row_nodes:
    for j in col_nodes:
        dist[i][j] = float(mat[ridx[i], cidx[j]])
customers = sorted(w.keys())
print(f"[INIT] start building columns for customers={len(customers)}", flush=True)
columns = build_initial_columns(customers, w, v, tw, dist, green_virtual)
if not columns:
    raise RuntimeError("No initial feasible columns.")
print(f"[INIT] customers={len(customers)}, initial_columns={len(columns)}", flush=True)
max_iter = MAX_ITER
for _it in range(max_iter):
    iter_id = _it + 1
    print(f"[CG] iter={iter_id} start, column_pool={len(columns)}", flush=True)
    rmp, pi_vec, sigma_vec = solve_rmp(columns, customers)
    print(f"[CG] iter={iter_id} rmp_obj={rmp.fun:.4f}", flush=True)
    pi = {c: float(pi_vec[idx]) for idx, c in enumerate(customers)}
    new_cols: List[Column] = []
    for veh in VEHICLES:
        cols_m = pricing_dynamic(
            veh,
            customers,
            pi,
            float(sigma_vec[veh.idx]),
            w,
            v,
            tw,
            dist,
            green_virtual,
        )
        print(
            f"[CG] iter={iter_id} vehicle={veh.name} priced_cols={len(cols_m)}",
            flush=True,
        )
        new_cols.extend(cols_m)
    added = 0
    existing_keys = {(c.veh_idx, tuple(c.route)) for c in columns}
    for col in new_cols:
        key = (col.veh_idx, tuple(col.route))
        if key in existing_keys:
            continue
        rc = col.cost - sum(pi[x] for x in col.route) - float(sigma_vec[col.veh_idx])
        if rc < -ADD_COL_RC_TOL:
            columns.append(col)
            existing_keys.add(key)
            added += 1
    print(
        f"[CG] iter={iter_id} new_candidates={len(new_cols)} added={added} pool={len(columns)}",
        flush=True,
    )
    if added == 0:
        print(f"[CG] iter={iter_id} stop: no improving columns", flush=True)
        break
    print("[FINAL] solving final RMP", flush=True)
    rmp, _, _ = solve_rmp(columns, customers)
    print("[FINAL] solving exact MILP recovery", flush=True)
    chosen_idx = integer_recover(columns, customers)
    chosen_cols = [columns[i] for i in chosen_idx]
    print(f"[FINAL] MILP selected_routes={len(chosen_cols)}", flush=True)
    route_rows = []
    arrival_rows = []
    usage = {veh.idx: 0 for veh in VEHICLES}
    total = {"startUp": 0.0, "energy": 0.0, "carbon": 0.0, "tw": 0.0, "sum": 0.0, "distance": 0.0}
    for rid, col in enumerate(chosen_cols, start=1):

```

```

veh = VEHICLES[col.veh_idx]
usage[veh.idx] += 1
ev = eval_route(col.route, veh, w, v, tw, dist, green_virtual)
total["startup"] += ev["startup_cost"]
total["energy"] += ev["energy_cost"]
total["carbon"] += ev["carbon_cost"]
total["tw"] += ev["tw_cost"]
total["sum"] += ev["total_cost"]
total["distance"] += ev["distance"]
route_rows.append(
    {
        "route_id": rid,
        "vehicle": veh.name,
        "virtual_nodes": "->".join(["0"] + [str(x) for x in col.route] + ["0"]),
        "virtual_ids": "->".join(["0#0"] + [form_id[x] for x in col.route] + ["0#0"]),
        "num_nodes": len(col.route),
        "departure_hour": ev["departure_hour"],
        "cost": ev["total_cost"],
        "startup": ev["startup_cost"],
        "energy": ev["energy_cost"],
        "carbon_cost": ev["carbon_cost"],
        "tw_cost": ev["tw_cost"],
        "distance_km": ev["distance"],
    }
)
t = float(ev["departure_hour"])
load_w = sum(w[c] for c in col.route)
load_v = sum(v[c] for c in col.route)
prev = 0
for c in col.route:
    d = dist[prev][c]
    tt, _, _ = leg_cost(veh, d, t, load_w)
    t += tt
    a, b = tw[c]
    before_w = load_w
    before_v = load_v
    after_w = before_w - w[c]
    after_v = before_v - v[c]
    arrival_rows.append(
        {
            "route_id": rid,
            "vehicle": veh.name,
            "virtual_customer": c,
            "virtual_id": form_id[c],
            "orig_customer": orig_id[c],
            "arrival_hour": round(t, 4),
            "tw_start_hour": a,
            "tw_end_hour": b,
            "early_h": max(a - t, 0.0),
            "late_h": max(t - b, 0.0),
            "in_green_zone": int(c in green_virtual),
            "load_weight_before_kg": round(before_w, 4),
            "deliver_weight_kg": round(w[c], 4),
            "load_weight_after_kg": round(after_w, 4),
            "load_volume_before_m3": round(before_v, 4),
            "deliver_volume_m3": round(v[c], 4),
            "load_volume_after_m3": round(after_v, 4),
        }
    )
    if t < a:
        t = a
    t += SERVICE_HOURS
    load_w = after_w
    load_v = after_v
    prev = c
usage_rows = [
    {"vehicle": veh.name, "used": usage[veh.idx], "limit": veh.count_limit} for veh in VEHICLES
]
pd.DataFrame(route_rows).to_excel(base + r"\q2_dw_solution_routes.xlsx", index=False)
pd.DataFrame(arrival_rows).to_excel(base + r"\q2_dw_solution_arrivals.xlsx", index=False)
pd.DataFrame(usage_rows).to_excel(base + r"\q2_dw_solution_vehicle_usage.xlsx", index=False)
map_df.to_excel(base + r"\q2_dw_solution_virtual_mapping.xlsx", index=False)
with open(base + r"\q2_dw_solution_summary.txt", "w", encoding="utf-8") as f:
    f.write("Q2 D-W + dynamic pricing with strict fuel green-zone pruning\n")
    f.write(f"Columns in pool: {len(columns)}\n")
    f.write(f"RMP LP objective: {rmp.fun:.2f}\n")
    f.write(f"Integer recovered routes: {len(chosen_cols)}\n")
    f.write(f"Vehicle usage:\n")
    for row in usage_rows:
        f.write(f"    {row['vehicle']}: {row['used']}/{row['limit']}\n")
    f.write(f"\nRecovered integer cost breakdown (RMB):\n")
    f.write(f"    Startup: {total['startup']:.2f}\n")
    f.write(f"    Energy: {total['energy']:.2f}\n")
    f.write(f"    Carbon: {total['carbon']:.2f}\n")
    f.write(f"    TimeWindow: {total['tw']:.2f}\n")

```

```

        f.write(f"    TOTAL: {total['sum']:.2f}\n")
        f.write(f"    Distance(km): {total['distance']:.2f}\n")
    print("Done.")
    print(base + r"\q2_dw_solution_summary.txt")
if __name__ == "__main__":
    main()

```

附录 4 问题三

```

import copy
import math
import random
import time
from collections import defaultdict
from dataclasses import dataclass
from pathlib import Path
from typing import Dict, List, Optional, Set, Tuple
import pandas as pd
SERVICE_HOURS = 20.0 / 60.0
STARTUP_COST = 400.0
EARLY_COST_PER_HOUR = 20.0
LATE_COST_PER_HOUR = 50.0
CARBON_COST_PER_KG = 0.65
BAN_END = 16.0
MIN_TENURE = 5
MAX_TENURE = 20
INIT_TENURE = 10
INIT_TEMP = 100.0
COOLING = 0.98
MAX_TABU_ITERS = 400
CANDIDATES_PER_ITER = 80
PENALTY_INFEAS = 1e6
REPEAT_SEEDS = [2026, 2027, 2028, 2029, 2030]
ROLL_DT_H = 0.5
ROLL_H_H = 2.0
DAY_START = 8.0
DAY_END = 20.0
def snap_time(t: float) -> float:
    return round(float(t), 4)
def periodic_decision_times() -> List[float]:
    out: List[float] = []
    t = DAY_START
    while t <= DAY_END + 1e-9:
        out.append(snap_time(t))
        t += ROLL_DT_H
    return out
@dataclass(frozen=True)
class VehicleType:
    idx: int
    name: str
    kind: str
    capacity_weight: float
    capacity_volume: float
    count_limit: int
    energy_price: float
    carbon_coeff: float
    full_load_increase: float
@dataclass
class RoutePlan:
    vehicle_idx: int
    customers: List[int]
@dataclass
class Event:
    t: float
    typ: str
    payload: Dict[str, object]
VEHICLES: List[VehicleType] = [
    VehicleType(0, "F3000", "fuel", 3000.0, 13.5, 60, 7.61, 2.547, 0.40),
    VehicleType(1, "F1500", "fuel", 1500.0, 10.8, 50, 7.61, 2.547, 0.40),
    VehicleType(2, "F1250", "fuel", 1250.0, 6.5, 50, 7.61, 2.547, 0.40),
    VehicleType(3, "E3000", "ev", 3000.0, 15.0, 10, 1.64, 0.501, 0.35),
    VehicleType(4, "E1250", "ev", 1250.0, 8.5, 15, 1.64, 0.501, 0.35),
]
def parse_hhmm_to_hour(x: str) -> float:
    h, m = str(x).split(":")
    return int(h) + int(m) / 60.0
def segments() -> List[Tuple[float, float, float]]:
    return [
        (8.0, 9.0, 9.8),
        (9.0, 10.0, 55.3),
        (10.0, 11.5, 35.4),
        (11.5, 13.0, 9.8),
        (13.0, 15.0, 55.3),
        (15.0, 17.0, 35.4),
    ]

```



```

        (17.0, 19.0, 9.8),
        (19.0, 20.0, 55.3),
    ]
def speed_at(t: float) -> float:
    x = t % 24.0
    for s, e, v in segments():
        if s <= x < e:
            return v
    return 35.4
def next_boundary(t: float) -> float:
    x = t % 24.0
    for s, e, _ in segments():
        if s <= x < e:
            return t + (e - x)
    return math.floor(t) + 1.0
def travel_profile(distance_km: float, depart: float) -> Tuple[float, List[Tuple[float, float]]]:
    remain = distance_km
    t = depart
    elapsed = 0.0
    segs: List[Tuple[float, float]] = []
    while remain > 1e-9:
        v = speed_at(t)
        b = next_boundary(t)
        dt = max(1e-6, b - t)
        can_go = v * dt
        if can_go >= remain:
            segs.append((remain, v))
            elapsed += remain / v
            return elapsed, segs
        segs.append((can_go, v))
        remain -= can_go
        elapsed += dt
        t += dt
    return elapsed, segs
def energy_per_100km(v: float, kind: str) -> float:
    if kind == "fuel":
        return 0.0025 * v * v - 0.2554 * v + 31.75
    return 0.0014 * v * v - 0.12 * v + 36.19
def dist_scaled(i: int, j: int, dist: List[List[float]], dist_scale: Dict[int, float]) -> float:
    si = dist_scale.get(i, 1.0)
    sj = dist_scale.get(j, 1.0)
    return dist[i][j] * 0.5 * (si + sj)
def leg_cost(
    veh: VehicleType,
    i: int,
    j: int,
    depart: float,
    load_w: float,
    dist: List[List[float]],
    dist_scale: Dict[int, float],
) -> Tuple[float, float, float, float]:
    d = dist_scaled(i, j, dist, dist_scale)
    tt, segs = travel_profile(d, depart)
    e0 = 0.0
    for seg_d, seg_v in segs:
        e0 += energy_per_100km(seg_v, veh.kind) * seg_d / 100.0
    load_ratio = min(1.0, max(0.0, load_w / veh.capacity_weight))
    e = e0 * (1.0 + veh.full_load_increase * load_ratio)
    e_cost = e * veh.energy_price
    c_cost = e * veh.carbon_coeff * CARBON_COST_PER_KG
    return tt, e_cost, c_cost, d
def eval_route(
    route: List[int],
    veh: VehicleType,
    w: Dict[int, float],
    v: Dict[int, float],
    tw: Dict[int, Tuple[float, float]],
    dist: List[List[float]],
    green_set: Set[int],
    dist_scale: Dict[int, float],
    start_t: float,
) -> Dict[str, float]:
    sw = sum(w[c] for c in route)
    sv = sum(v[c] for c in route)
    if sw > veh.capacity_weight + 1e-9 or sv > veh.capacity_volume + 1e-9:
        return {"feasible": 0.0, "total_cost": 1e30}
    t = start_t
    load_w = sw
    startup = STARTUP_COST
    energy = 0.0
    carbon = 0.0
    tw_cost = 0.0
    dist_sum = 0.0
    late_cnt = 0.0
    prev = 0

```

```

for c in route:
    tt, ec, cc, d = leg_cost(veh, prev, c, t, load_w, dist, dist_scale)
    t += tt
    dist_sum += d
    if veh.kind == "fuel" and c in green_set and t < BAN_END:
        t = BAN_END
    energy += ec
    carbon += cc
    a, b = tw[c]
    early = max(a - t, 0.0)
    late = max(t - b, 0.0)
    if late > 1e-9:
        late_cnt += 1.0
    tw_cost += EARLY_COST_PER_HOUR * early + LATE_COST_PER_HOUR * late
    if t < a:
        t = a
    t += SERVICE_HOURS
    load_w -= w[c]
    prev = c
tt, ec, cc, d = leg_cost(veh, prev, 0, t, load_w, dist, dist_scale)
_ = tt
dist_sum += d
energy += ec
carbon += cc
total = startup + energy + carbon + tw_cost
return {
    "feasible": 1.0,
    "total_cost": total,
    "startup_cost": startup,
    "energy_cost": energy,
    "carbon_cost": carbon,
    "tw_cost": tw_cost,
    "distance": dist_sum,
    "late_cnt": late_cnt,
}
}

def parse_routes_from_file(routes_path: Path) -> List[RoutePlan]:
    df = pd.read_excel(routes_path)
    out: List[RoutePlan] = []
    veh_name_to_idx = {x.name: x.idx for x in VEHICLES}
    for _, row in df.iterrows():
        nodes_txt = str(row["virtual_nodes"])
        seq = [int(x) for x in nodes_txt.split("->") if x.strip().isdigit()]
        if len(seq) < 3:
            continue
        customers = seq[1:-1]
        vname = str(row["vehicle"])
        if vname not in veh_name_to_idx:
            continue
        out.append(RoutePlan(vehicle_idx=veh_name_to_idx[vname], customers=customers))
    return out

def extract_served_until(
    routes: List[RoutePlan],
    w: Dict[int, float],
    v: Dict[int, float],
    tw: Dict[int, Tuple[float, float]],
    dist: List[List[float]],
    green_set: Set[int],
    dist_scale: Dict[int, float],
    start_t: float,
    t_cut: float,
) -> Set[int]:
    served: Set[int] = set()
    for rp in routes:
        veh = VEHICLES[rp.vehicle_idx]
        t = start_t
        load_w = sum(w[c] for c in rp.customers)
        prev = 0
        for c in rp.customers:
            tt, _, _ = leg_cost(veh, prev, c, t, load_w, dist, dist_scale)
            t += tt
            if veh.kind == "fuel" and c in green_set and t < BAN_END:
                t = BAN_END
            a, _ = tw[c]
            if t < a:
                t = a
            if t <= t_cut + 1e-9:
                served.add(c)
            t += SERVICE_HOURS
            load_w -= w[c]
            prev = c
        return served

def build_remaining_routes(previous: List[RoutePlan], remaining: Set[int]) -> List[RoutePlan]:
    keep: List[RoutePlan] = []
    assigned: Set[int] = set()
    for rp in previous:

```

```

nxt = [c for c in rp.customers if c in remaining]
if nxt:
    keep.append(RoutePlan(vehicle_idx=rp.vehicle_idx, customers=nxt))
    assigned.update(nxt)
unassigned = sorted(list(remaining - assigned))
if not unassigned:
    return keep
# Append as singleton to first feasible vehicle by capacity priority.
veh_order = [0, 3, 1, 4, 2]
veh_use = {x.idx: 0 for x in VEHICLES}
for r in keep:
    veh_use[r.vehicle_idx] += 1
for c in unassigned:
    chosen: Optional[int] = None
    for vid in veh_order:
        veh = VEHICLES[vid]
        if veh_use[vid] >= veh.count_limit:
            continue
        # Capacity checked at route evaluation stage; singleton always feasible in this dataset.
        chosen = vid
        break
    if chosen is None:
        chosen = 0
    keep.append(RoutePlan(vehicle_idx=chosen, customers=[c]))
    veh_use[chosen] += 1
return keep
def solution_cost(
    routes: List[RoutePlan],
    w: Dict[int, float],
    v: Dict[int, float],
    tw: Dict[int, Tuple[float, float]],
    dist: List[List[float]],
    green_set: Set[int],
    dist_scale: Dict[int, float],
    start_t: float,
) -> Dict[str, float]:
    total = {"cost": 0.0, "startup": 0.0, "energy": 0.0, "carbon": 0.0, "tw": 0.0, "distance": 0.0, "late_cnt": 0.0}
    veh_use = {x.idx: 0 for x in VEHICLES}
    covered: Set[int] = set()
    duplicate = 0
    for rp in routes:
        veh_use[rp.vehicle_idx] += 1
        for c in rp.customers:
            if c in covered:
                duplicate += 1
            covered.add(c)
    ev = eval_route(rp.customers, VEHICLES[rp.vehicle_idx], w, v, tw, dist, green_set, dist_scale, start_t)
    if ev["feasible"] < 0.5:
        total["cost"] += PENALTY_INFEAS
        continue
    total["cost"] += ev["total_cost"]
    total["startup"] += ev["startup_cost"]
    total["energy"] += ev["energy_cost"]
    total["carbon"] += ev["carbon_cost"]
    total["tw"] += ev["tw_cost"]
    total["distance"] += ev["distance"]
    total["late_cnt"] += ev["late_cnt"]
    overflow = 0
    for veh in VEHICLES:
        if veh_use[veh.idx] > veh.count_limit:
            overflow += veh_use[veh.idx] - veh.count_limit
    if overflow > 0:
        total["cost"] += overflow * PENALTY_INFEAS
    if duplicate > 0:
        total["cost"] += duplicate * PENALTY_INFEAS
    return total
def clone_routes(routes: List[RoutePlan]) -> List[RoutePlan]:
    return [RoutePlan(vehicle_idx=r.vehicle_idx, customers=list(r.customers)) for r in routes]
def movable_within_horizon(remaining: Set[int], tw: Dict[int, Tuple[float, float]], t: float, H: float) -> Set[int]:
    """Customers whose time-window end falls within planning horizon [t, t+H]."""
    m = {c for c in remaining if tw[c][1] <= t + H + 1e-9}
    if not m:
        return set(remaining)
    return m
def candidate_moves(
    routes: List[RoutePlan],
    rng: random.Random,
    w: Dict[int, float],
    v: Dict[int, float],
    movable: Optional[Set[int]] = None,
) -> List[Tuple[List[RoutePlan], Tuple[object, ...]]]:
    moves: List[Tuple[List[RoutePlan], Tuple[object, ...]]] = []
    r_ids = [i for i, r in enumerate(routes) if r.customers]
    if not r_ids:
        return moves

```

```

for _ in range(12):
    rid = rng.choice(r_ids)
    route = routes[rid]
    if len(route.customers) < 2:
        continue
    i = rng.randrange(len(route.customers))
    j = rng.randrange(len(route.customers))
    if i == j:
        continue
    cid = route.customers[i]
    if movable is not None and cid not in movable:
        continue
    nr = clone_routes(routes)
    c = nr[rid].customers.pop(i)
    nr[rid].customers.insert(j, c)
    moves.append((nr, ("intra", rid, c, i, j)))
if len(r_ids) >= 2:
    for _ in range(16):
        r1, r2 = rng.sample(r_ids, 2)
        if not routes[r1].customers:
            continue
        i = rng.randrange(len(routes[r1].customers))
        j = rng.randrange(len(routes[r2].customers) + 1)
        cid = routes[r1].customers[i]
        if movable is not None and cid not in movable:
            continue
        nr = clone_routes(routes)
        c = nr[r1].customers.pop(i)
        nr[r2].customers.insert(j, c)
        w1 = sum(w[x] for x in nr[r1].customers) if nr[r1].customers else 0.0
        v1 = sum(v[x] for x in nr[r1].customers) if nr[r1].customers else 0.0
        w2 = sum(w[x] for x in nr[r2].customers)
        v2 = sum(v[x] for x in nr[r2].customers)
        if w2 > VEHICLES[nr[r2].vehicle_idx].capacity_weight + 1e-9 or v2 > VEHICLES[nr[r2].vehicle_idx].capacity_volume + 1e-9:
            continue
        if nr[r1].customers:
            if w1 > VEHICLES[nr[r1].vehicle_idx].capacity_weight + 1e-9 or v1 > VEHICLES[nr[r1].vehicle_idx].capacity_volume + 1e-9:
                continue
        if not nr[r1].customers:
            nr.pop(r1)
        moves.append((nr, ("inter_reloc", r1, r2, c)))
if len(r_ids) >= 2:
    for _ in range(12):
        r1, r2 = rng.sample(r_ids, 2)
        if not routes[r1].customers or not routes[r2].customers:
            continue
        i = rng.randrange(len(routes[r1].customers))
        j = rng.randrange(len(routes[r2].customers))
        c1 = routes[r1].customers[i]
        c2 = routes[r2].customers[j]
        if movable is not None and (c1 not in movable or c2 not in movable):
            continue
        nr = clone_routes(routes)
        nr[r1].customers[i], nr[r2].customers[j] = nr[r2].customers[j], nr[r1].customers[i]
        w1 = sum(w[x] for x in nr[r1].customers)
        v1 = sum(v[x] for x in nr[r1].customers)
        w2 = sum(w[x] for x in nr[r2].customers)
        v2 = sum(v[x] for x in nr[r2].customers)
        if w1 <= VEHICLES[nr[r1].vehicle_idx].capacity_weight + 1e-9 and v1 <= VEHICLES[nr[r1].vehicle_idx].capacity_volume + 1e-9:
            if w2 <= VEHICLES[nr[r2].vehicle_idx].capacity_weight + 1e-9 and v2 <= VEHICLES[nr[r2].vehicle_idx].capacity_volume + 1e-9:
                moves.append((nr, ("inter_swap", c1, c2)))
    rng.shuffle(moves)
return moves[:CANDIDATES_PER_ITER]
def tabu_optimize(
    routes_init: List[RoutePlan],
    w: Dict[int, float],
    v: Dict[int, float],
    tw: Dict[int, Tuple[float, float]],
    dist: List[List[float]],
    green_set: Set[int],
    dist_scale: Dict[int, float],
    start_t: float,
    seed: int,
    movable: Optional[Set[int]] = None,
) -> Tuple[List[RoutePlan], Dict[str, float]]:
    rng = random.Random(seed)
    current = clone_routes(routes_init)
    cur_eval = solution_cost(current, w, v, tw, dist, green_set, dist_scale, start_t)
    best = clone_routes(current)
    best_eval = dict(cur_eval)
    tabu_until: Dict[Tuple[object, ...], int] = {}
    tenure = INIT_TENURE
    temp = INIT_TEMP
    no_imp = 0
    for it in range(1, MAX_TABU_ITERS + 1):

```

```

cand_moves = candidate_moves(current, rng, w, v, movable)
if not cand_moves:
    break
chosen_sol: Optional[List[RoutePlan]] = None
chosen_key: Optional[Tuple[object, ...]] = None
chosen_eval: Optional[Dict[str, float]] = None
for sol, mkey in cand_moves:
    ev = solution_cost(sol, w, v, tw, dist, green_set, dist_scale, start_t)
    is_tabu = tabu_until.get(mkey, -1) >= it
    # Aspiration: allow tabu if beats global best.
    if is_tabu and ev["cost"] >= best_eval["cost"] - 1e-9:
        continue
    if chosen_eval is None or ev["cost"] < chosen_eval["cost"]:
        chosen_sol = sol
        chosen_key = mkey
        chosen_eval = ev
if chosen_sol is None or chosen_key is None or chosen_eval is None:
    continue
delta = chosen_eval["cost"] - cur_eval["cost"]
accept = False
if delta <= 0:
    accept = True
else:
    p = math.exp(-delta / max(temp, 1e-6))
    if rng.random() <= p:
        accept = True
if accept:
    current = chosen_sol
    cur_eval = chosen_eval
    tabu_until[chosen_key] = it + tenure
    if cur_eval["cost"] < best_eval["cost"] - 1e-9:
        best = clone_routes(current)
        best_eval = dict(cur_eval)
        tenure = max(MIN_TENURE, tenure - 1)
        no_imp = 0
    else:
        tenure = min(MAX_TENURE, tenure + 1)
        no_imp += 1
    temp *= COOLING
    if no_imp > 40:
        temp = max(temp, 20.0)
return best, best_eval
def build_scenarios(active_pool: List[int], reserve_pool: List[int], green_set: Set[int], seed: int) -> Dict[str, List[Event]]:
    rng = random.Random(seed)
    non_green_reserve = [x for x in reserve_pool if x not in green_set]
    green_reserve = [x for x in reserve_pool if x in green_set]
    non_green_active = [x for x in active_pool if x not in green_set]
    if len(non_green_reserve) < 6:
        raise RuntimeError("Reserve pool too small for synthetic new-order events.")
    s_a = [
        Event(10.5, "add", {"customer": non_green_reserve[0]}),
        Event(14.0, "cancel", {"customer": rng.choice(non_green_active)}),
    ]
    s_b = [
        Event(9.6667, "add", {"customer": non_green_reserve[1]}),
        Event(9.6667, "add", {"customer": non_green_reserve[2]}),
        Event(12.5, "addr_change", {"customer": rng.choice(non_green_active), "scale": 1.25}),
        Event(15.3333, "tw_tighten", {"customer": rng.choice(non_green_active), "shrink_h": 0.5}),
    ]
    g_add = green_reserve[0] if green_reserve else reserve_pool[0]
    s_c = [
        Event(10.0, "add", {"customer": g_add}),
        Event(10.0, "add", {"customer": non_green_reserve[3]}),
        Event(10.0, "add", {"customer": non_green_reserve[4]}),
        Event(11.5, "cancel", {"customer": rng.choice(non_green_active)}),
        Event(13.1667, "addr_change", {"customer": rng.choice(non_green_active), "scale": 1.15}),
        Event(13.1667, "addr_change", {"customer": rng.choice(non_green_active), "scale": 1.10}),
        Event(16.0, "tw_tighten", {"customer": rng.choice(non_green_active), "shrink_h": 0.40}),
        Event(16.0, "tw_tighten", {"customer": rng.choice(non_green_active), "shrink_h": 0.30}),
    ]
    return {"A_light": s_a, "B_medium": s_b, "C_heavy": s_c}
def scenario_run(
    name: str,
    events: List[Event],
    base_routes: List[RoutePlan],
    all_customers: Set[int],
    w: Dict[int, float],
    v: Dict[int, float],
    tw_origin: Dict[int, Tuple[float, float]],
    dist: List[List[float]],
    green_set: Set[int],
    seed: int,
) -> Tuple[Dict[str, float], List[Dict[str, object]], List[RoutePlan]]:
    """Rolling horizon + event-driven: replan at each t in periodic grid ∪ event times."""
    w = copy.deepcopy(w)

```

```

v = copy.deepcopy(v)
tw = copy.deepcopy(tw_origin)
dist_scale: Dict[int, float] = {}
for c in all_customers:
    dist_scale[c] = 1.0
active = set(all_customers)
routes = clone_routes(base_routes)
logs: List[Dict[str, object]] = []
total_reopt_sec = 0.0
ev_by_t: Dict[float, List[Event]] = defaultdict(list)
for ev in events:
    ev_by_t[snap_time(ev.t)].append(ev)
decision_times = sorted(set(periodic_decision_times()) | set(ev_by_t.keys()))
n_steps = len(decision_times)
for idx, t_dec in enumerate(decision_times, start=1):
    served = extract_served_until(routes, w, v, tw, dist, green_set, dist_scale, 8.0, t_dec)
    remaining = {c for c in active if c not in served}
    evs_here = sorted(ev_by_t.get(t_dec, []), key=lambda e: e.typ)
    for ev in evs_here:
        if ev.typ == "add":
            c = int(ev.payload["customer"])
            if c not in served:
                if "weight" in ev.payload:
                    w[c] = float(ev.payload["weight"])
                if "volume" in ev.payload:
                    v[c] = float(ev.payload["volume"])
                dist_scale.setdefault(c, 1.0)
                remaining.add(c)
                active.add(c)
            elif ev.typ == "cancel":
                c = int(ev.payload["customer"])
                remaining.discard(c)
                active.discard(c)
            elif ev.typ == "addr_change":
                c = int(ev.payload["customer"])
                if c in remaining:
                    j = int(ev.payload.get("neighbor", c))
                    if j != c and 0 <= j < len(dist):
                        base = dist[0][c] + dist[c][0] + 1e-6
                        alt = dist[0][j] + dist[j][0]
                        dist_scale[c] = max(0.85, min(1.35, alt / base))
                    else:
                        dist_scale[c] = float(ev.payload.get("scale", 1.2))
            elif ev.typ == "tw_tighten":
                c = int(ev.payload["customer"])
                if c in remaining:
                    a, b = tw[c]
                    shrink = float(ev.payload.get("shrink_h", 0.5))
                    tw[c] = (a, max(a + 0.5, b - shrink))
    warm = build_remaining_routes(routes, remaining)
    movable = movable_within_horizon(remaining, tw, t_dec, ROLL_H_H)
    t0 = time.perf_counter()
    opt_routes, opt_eval = tabu_optimize(
        warm,
        w,
        v,
        tw,
        dist,
        green_set,
        dist_scale,
        start_t=max(8.0, t_dec),
        seed=seed + idx * 13,
        movable=movable,
    )
    dt = time.perf_counter() - t0
    total_reopt_sec += dt
    routes = opt_routes
    trig = "rolling_periodic"
    if evs_here:
        trig = "event_or_mixed"
    typ_str = ",".join(ev.typ for ev in evs_here if evs_here else "-")
    aff = [int(ev.payload.get("customer", -1)) for ev in evs_here]
    logs.append(
        {
            "scenario": name,
            "step_idx": idx,
            "decision_time": t_dec,
            "trigger": trig,
            "event_types": typ_str,
            "affected_customers": ",".join(str(x) for x in aff) if aff else "",
            "horizon_h_h": ROLL_H_H,
            "roll_dt_h": ROLL_DT_H,
            "movable_n": len(movable),
            "remaining_n": len(remaining),
            "reopt_time_s": round(dt, 4),
        }
    )

```

```

        "cost_after_reopt": round(opt_eval["cost"], 2),
    }
)
required_final = set(active)
full_day_warm = build_remaining_routes(base_routes, required_final)
full_day_routes, _ = tabu_optimize(
    full_day_warm,
    w,
    v,
    tw,
    dist,
    green_set,
    dist_scale,
    start_t=DAY_START,
    seed=seed + 99991,
    movable=None,
)
final_eval = solution_cost(full_day_routes, w, v, tw, dist, green_set, dist_scale, start_t=DAY_START)
total_stops = len(required_final)
otd = 1.0
if total_stops > 0:
    otd = max(0.0, 1.0 - final_eval["late_cnt"] / total_stops)
unit_service_cost = final_eval["cost"] / max(total_stops, 1)
summary = {
    "scenario": name,
    "dynamic_events": len(events),
    "replan_steps": n_steps,
    "expected_orders": len(required_final),
    "served_orders": total_stops,
    "served_gap": total_stops - len(required_final),
    "final_cost": round(final_eval["cost"], 2),
    "unit_service_cost": round(unit_service_cost, 4),
    "startup": round(final_eval["startup"], 2),
    "energy": round(final_eval["energy"], 2),
    "carbon": round(final_eval["carbon"], 2),
    "tw_penalty": round(final_eval["tw"], 2),
    "distance_km": round(final_eval["distance"], 2),
    "otd_rate": round(otd, 4),
    "avg_reopt_time_s": round(total_reopt_sec / max(n_steps, 1), 4),
    "routes": len(full_day_routes),
}
return summary, logs, full_day_routes
def main() -> None:
    base = Path(r"c:\Users\HP\Desktop\有用")
    order_df = pd.read_excel(base / "q1_preprocessed_订单信息_形式客户.xlsx")
    tw_df = pd.read_excel(base / "q1_preprocessed_时间窗_形式客户.xlsx")
    dist_df = pd.read_excel(base / "q1_preprocessed_距离矩阵_形式客户.xlsx")
    map_df = pd.read_excel(base / "q1_preprocessed_映射表.xlsx")
    coords_df = pd.read_excel(base / "q1_preprocessed_客户坐标_形式客户.xlsx")
    q2_routes_file = base / "q2_dw_solution_routes.xlsx"
    if not q2_routes_file.exists():
        raise FileNotFoundError("Need q2_dw_solution_routes.xlsx as initial plan.")
    w = {int(r["客户编号(新)"]): float(r["总重量"]) for _, r in order_df.iterrows()}
    v = {int(r["客户编号(新)"]): float(r["总体积"]) for _, r in order_df.iterrows()}
    tw = {
        int(r["客户编号(新)"]): (
            parse_hhmm_to_hour(r["开始时间"]),
            parse_hhmm_to_hour(r["结束时间"]),
        )
        for _, r in tw_df.iterrows()
    }
    row_nodes = dist_df.iloc[:, 0].astype(int).tolist()
    col_nodes = [int(x) for x in dist_df.columns[1:]]
    mat = dist_df.iloc[:, 1:].to_numpy(float)
    ridx = {n: i for i, n in enumerate(row_nodes)}
    cidx = {n: i for i, n in enumerate(col_nodes)}
    nmax = max(max(row_nodes), max(col_nodes))
    dist = [[0.0 for _ in range(nmax + 1)] for _ in range(nmax + 1)]
    for i in row_nodes:
        for j in col_nodes:
            dist[i][j] = float(mat[ridx[i], cidx[j]])
    coords = coords_df.iloc[:, :4].copy()
    coords.columns = ["类型", "ID", "X", "Y"]
    non_depot = coords[coords["类型"] != "配送中心"].copy()
    non_depot["in_zone"] = non_depot["X"] ** 2 + non_depot["Y"] ** 2 <= 100.0 + 1e-9
    green_orig = set(non_depot.loc[non_depot["in_zone"], "ID"].astype(int).tolist())
    green_virtual = set(
        int(r["新客户编号"]) for _, r in map_df.iterrows() if int(r["原客户编号"]) in green_orig
    )
    base_routes = parse_routes_from_file(q2_routes_file)
    all_customers = sorted(list(w.keys()))
    summary_rows: List[Dict[str, object]] = []
    log_rows: List[Dict[str, object]] = []
    final_routes_rows: List[Dict[str, object]] = []
    per_run_rows: List[Dict[str, object]] = []

```

```

for run_seed in REPEAT_SEEDS:
    rng = random.Random(run_seed)
    reserve_pool = rng.sample(all_customers, 12)
    active_pool = [c for c in all_customers if c not in reserve_pool]
    all_active = set(active_pool)
    run_base_routes = build_remaining_routes(base_routes, all_active)
    base_eval = solution_cost(
        run_base_routes,
        w,
        v,
        tw,
        dist,
        green_virtual,
        {c: 1.0 for c in all_customers},
        8.0,
    )
    base_served = sum(len(r.customers) for r in run_base_routes)
    base_unit_cost = base_eval["cost"] / max(base_served, 1)
    scenarios = build_scenarios(active_pool, reserve_pool, green_virtual, seed=run_seed)
    per_run_rows.append(
        {
            "seed": run_seed,
            "scenario": "S0_baseline",
            "dynamic_events": 0,
            "replan_steps": 0,
            "served_orders": base_served,
            "final_cost": round(base_eval["cost"], 2),
            "unit_service_cost": round(base_unit_cost, 4),
            "startup": round(base_eval["startup"], 2),
            "energy": round(base_eval["energy"], 2),
            "carbon": round(base_eval["carbon"], 2),
            "tw_penalty": round(base_eval["tw"], 2),
            "distance_km": round(base_eval["distance"], 2),
            "otd_rate": round(max(0.0, 1.0 - base_eval["late_cnt"] / max(len(all_active), 1)), 4),
            "avg_reopt_time_s": 0.0,
            "routes": len(run_base_routes),
            "cost_loss_rate_vs_s0": 0.0,
            "unit_cost_index_vs_s0": 1.0,
            "composite_score": round(0.7 * 1.0 + 0.3 * max(0.0, 1.0 - base_eval["late_cnt"] / max(len(all_active), 1)), 4),
        }
    )
for s_idx, (name, events) in enumerate(scenarios.items(), start=1):
    s_summary, s_logs, s_routes = scenario_run(
        name=name,
        events=events,
        base_routes=run_base_routes,
        all_customers=all_active,
        w=w,
        v=v,
        tw_origin=tw,
        dist=dist,
        green_set=green_virtual,
        seed=run_seed + 100 * s_idx,
    )
    s_summary["seed"] = run_seed
    s_summary["cost_loss_rate_vs_s0"] = round(
        (s_summary["final_cost"] - base_eval["cost"]) / max(base_eval["cost"], 1e-9),
        4,
    )
    unit_idx = base_unit_cost / max(float(s_summary["unit_service_cost"]), 1e-9)
    s_summary["unit_cost_index_vs_s0"] = round(unit_idx, 4)
    s_summary["composite_score"] = round(0.7 * unit_idx + 0.3 * float(s_summary["otd_rate"]), 4)
    per_run_rows.append(s_summary)
    for x in s_logs:
        x["seed"] = run_seed
        log_rows.append(x)
    for rid, rp in enumerate(s_routes, start=1):
        final_routes_rows.append(
            {
                "seed": run_seed,
                "scenario": name,
                "route_id": rid,
                "vehicle": VEHICLES[rp.vehicle_idx].name,
                "virtual_nodes": "->".join(["0"] + [str(x) for x in rp.customers] + ["0"]),
                "num_nodes": len(rp.customers),
            }
        )
per_run_df = pd.DataFrame(per_run_rows)
agg_df = (
    per_run_df.groupby("scenario", as_index=False)
    .agg(
        runs=("seed", "count"),
        dynamic_events_mean=("dynamic_events", "mean"),
        replan_steps_mean=("replan_steps", "mean"),
        served_orders_mean=("served_orders", "mean"),
    )

```



```

        final_cost_mean=("final_cost", "mean"),
        final_cost_std=("final_cost", "std"),
        unit_service_cost_mean=("unit_service_cost", "mean"),
        unit_service_cost_std=("unit_service_cost", "std"),
        otd_rate_mean=("otd_rate", "mean"),
        otd_rate_std=("otd_rate", "std"),
        avg_reopt_time_s_mean=("avg_reopt_time_s", "mean"),
        avg_reopt_time_s_std=("avg_reopt_time_s", "std"),
        cost_loss_rate_vs_s0_mean=("cost_loss_rate_vs_s0", "mean"),
        cost_loss_rate_vs_s0_std=("cost_loss_rate_vs_s0", "std"),
        unit_cost_index_vs_s0_mean=("unit_cost_index_vs_s0", "mean"),
        unit_cost_index_vs_s0_std=("unit_cost_index_vs_s0", "std"),
        composite_score_mean=("composite_score", "mean"),
        composite_score_std=("composite_score", "std"),
    )
    .sort_values("scenario")
)
summary_df = agg_df.copy()
logs_df = pd.DataFrame(log_rows)
routes_df = pd.DataFrame(final_routes_rows)
summary_df.to_excel(base / "q3_dynamic_tabu_summary.xlsx", index=False)
per_run_df.to_excel(base / "q3_dynamic_tabu_per_run.xlsx", index=False)
logs_df.to_excel(base / "q3_dynamic_tabu_event_logs.xlsx", index=False)
routes_df.to_excel(base / "q3_dynamic_tabu_final_routes.xlsx", index=False)
with open(base / "q3_dynamic_tabu_summary.txt", "w", encoding="utf-8") as f:
    f.write("Q3 Rolling-horizon + event-driven dynamic dispatch (tabu search)\n")
    f.write(f"ROLL_DT={ROLL_DT_H}h, ROLL_H={ROLL_H_H}h (movable = tw_end<=t+H; else full set)\n")
    f.write(f"Repeated seeds: {REPEAT_SEEDS}\n")
    f.write("Reported as mean ± std across runs\n")
    f.write("Composite score = 0.7 * unit_cost_index_vs_s0 + 0.3 * otd_rate\n\n")
    for _, r in summary_df.iterrows():
        f.write(
            f"[{r['scenario']}] unit_cost={r['unit_service_cost_mean']:.2f}±{(0.0 if pd.isna(r['unit_service_cost_std']) else
r['unit_service_cost_std']):.2f}, "
            f"otd=({float(r['otd_rate_mean']) * 100:.2f})%±{(0.0 if pd.isna(r['otd_rate_std']) else float(r['otd_rate_std']) *
100):.2f})%", "
            f"score={r['composite_score_mean']:.4f}±{(0.0 if pd.isna(r['composite_score_std']) else r['composite_score_std']):.4f},
"
            f"avg_reopt={r['avg_reopt_time_s_mean']:.2f}±{(0.0 if pd.isna(r['avg_reopt_time_s_std']) else
r['avg_reopt_time_s_std']):.2f}s\n"
        )
    print("Done.")
    print(str(base / "q3_dynamic_tabu_summary.txt"))
if __name__ == "__main__":
    main()

```

附录5 灵敏度检验与误差分析

```

import math
from dataclasses import dataclass
from pathlib import Path
from typing import Dict, List, Tuple
import matplotlib.pyplot as plt
import pandas as pd
DATA_DIR = Path(r"c:\Users\HP\Desktop\有用")
ROUTE_FILE = Path(r"c:\Users\HP\Desktop\q1_dw_solution_routes.xlsx")
OUTPUT_DIR = DATA_DIR
SERVICE_HOURS = 20.0 / 60.0
STARTUP_COST = 400.0
@dataclass(frozen=True)
class VehicleType:
    idx: int
    name: str
    kind: str
    capacity_weight: float
    capacity_volume: float
    count_limit: int
    energy_price: float
    carbon_coeff: float
    full_load_increase: float
VEHICLES: List[VehicleType] = [
    VehicleType(0, "F3000", "fuel", 3000.0, 13.5, 60, 7.61, 2.547, 0.40),
    VehicleType(1, "F1500", "fuel", 1500.0, 10.8, 50, 7.61, 2.547, 0.40),
    VehicleType(2, "F1250", "fuel", 1250.0, 6.5, 50, 7.61, 2.547, 0.40),
    VehicleType(3, "E3000", "ev", 3000.0, 15.0, 10, 1.64, 0.501, 0.35),
    VehicleType(4, "E1250", "ev", 1250.0, 8.5, 15, 1.64, 0.501, 0.35),
]
VEH_BY_NAME = {v.name: v for v in VEHICLES}
def parse_hhmm_to_hour(x: str) -> float:
    h, m = str(x).split(":")
    return int(h) + int(m) / 60.0
def segments() -> List[Tuple[float, float, float]]:
    return [
        (8.0, 9.0, 9.8),

```

```

        (9.0, 10.0, 55.3),
        (10.0, 11.5, 35.4),
        (11.5, 13.0, 9.8),
        (13.0, 15.0, 55.3),
        (15.0, 17.0, 35.4),
        (17.0, 19.0, 9.8),
        (19.0, 20.0, 55.3),
    ]
def speed_at(t: float) -> float:
    x = t % 24.0
    for s, e, v in segments():
        if s <= x < e:
            return v
    return 35.4
def next_boundary(t: float) -> float:
    x = t % 24.0
    for s, e, _ in segments():
        if s <= x < e:
            return t + (e - x)
    return math.floor(t) + 1.0
def travel_profile(distance_km: float, depart: float) -> Tuple[float, List[Tuple[float, float]]]:
    remain = distance_km
    t = depart
    elapsed = 0.0
    segs: List[Tuple[float, float]] = []
    while remain > 1e-9:
        v = speed_at(t)
        b = next_boundary(t)
        dt = max(1e-6, b - t)
        can_go = v * dt
        if can_go >= remain:
            segs.append((remain, v))
            elapsed += remain / v
            return elapsed, segs
        segs.append((can_go, v))
        remain -= can_go
        elapsed += dt
        t += dt
    return elapsed, segs
def energy_per_100km(v: float, kind: str) -> float:
    if kind == "fuel":
        return 0.0025 * v * v - 0.2554 * v + 31.75
    return 0.0014 * v * v - 0.12 * v + 36.19
def leg_cost(
    veh: VehicleType,
    distance_km: float,
    depart: float,
    load_w: float,
    fuel_price_factor: float,
    elec_price_factor: float,
    carbon_price_factor: float,
) -> Tuple[float, float, float]:
    tt, segs = travel_profile(distance_km, depart)
    base_e = 0.0
    for seg_d, seg_v in segs:
        base_e += energy_per_100km(seg_v, veh.kind) * seg_d / 100.0
    ratio = min(1.0, max(0.0, load_w / veh.capacity_weight))
    e = base_e * (1.0 + veh.full_load_increase * ratio)
    energy_price = veh.energy_price * (fuel_price_factor if veh.kind == "fuel" else elec_price_factor)
    e_cost = e * energy_price
    c_cost = e * veh.carbon_coeff * 0.65 * carbon_price_factor
    return tt, e_cost, c_cost
def eval_route_with_params(
    route: List[int],
    veh: VehicleType,
    departure_hour: float,
    w: Dict[int, float],
    v: Dict[int, float],
    tw: Dict[int, Tuple[float, float]],
    dist: List[List[float]],
    fuel_price_factor: float = 1.0,
    elec_price_factor: float = 1.0,
    carbon_price_factor: float = 1.0,
    early_cost_factor: float = 1.0,
    late_cost_factor: float = 1.0,
) -> Dict[str, float]:
    sw = sum(w[c] for c in route)
    sv = sum(v[c] for c in route)
    if sw > veh.capacity_weight + 1e-9 or sv > veh.capacity_volume + 1e-9:
        return {"feasible": 0.0, "total_cost": 1e30}
    t = departure_hour
    load_w = sw
    startup = STARTUP_COST
    energy = 0.0
    carbon = 0.0

```

```

tw_cost = 0.0
dist_sum = 0.0
prev = 0
for c in route:
    d = dist[prev][c]
    dist_sum += d
    tt, ec, cc = leg_cost(
        veh,
        d,
        t,
        load_w,
        fuel_price_factor=fuel_price_factor,
        elec_price_factor=elec_price_factor,
        carbon_price_factor=carbon_price_factor,
    )
    t += tt
    energy += ec
    carbon += cc
    a, b = tw[c]
    early = max(a - t, 0.0)
    late = max(t - b, 0.0)
    tw_cost += 20.0 * early * early_cost_factor + 50.0 * late * late_cost_factor
    if t < a:
        t = a
    t += SERVICE_HOURS
    load_w -= w[c]
    prev = c
d_back = dist[prev][0]
dist_sum += d_back
tt, ec, cc = leg_cost(
    veh,
    d_back,
    t,
    load_w,
    fuel_price_factor=fuel_price_factor,
    elec_price_factor=elec_price_factor,
    carbon_price_factor=carbon_price_factor,
)
_ = tt
energy += ec
carbon += cc
total = startup + energy + carbon + tw_cost
return {
    "feasible": 1.0,
    "startup": startup,
    "energy": energy,
    "carbon": carbon,
    "tw_cost": tw_cost,
    "distance": dist_sum,
    "total_cost": total,
}

def read_inputs() -> Tuple[Dict[int, float], Dict[int, float], Dict[int, Tuple[float, float]], List[List[float]]]:
    order_df = pd.read_excel(DATA_DIR / "q1_preprocessed_订单信息_形式客户.xlsx")
    tw_df = pd.read_excel(DATA_DIR / "q1_preprocessed_时间窗_形式客户.xlsx")
    dist_df = pd.read_excel(DATA_DIR / "q1_preprocessed_距离矩阵_形式客户.xlsx")
    w = {int(r["客户编号(新)"]): float(r["总重量"]) for _, r in order_df.iterrows()}
    v = {int(r["客户编号(新)"]): float(r["总体积"]) for _, r in order_df.iterrows()}
    tw = {
        int(r["客户编号(新)"]): (parse_hhmm_to_hour(r["开始时间"]), parse_hhmm_to_hour(r["结束时间"]))
        for _, r in tw_df.iterrows()
    }

    row_nodes = dist_df.iloc[:, 0].astype(int).tolist()
    col_nodes = [int(x) for x in dist_df.columns[1:]]
    mat = dist_df.iloc[:, 1:].to_numpy(float)
    ridx = {n: i for i, n in enumerate(row_nodes)}
    cidx = {n: i for i, n in enumerate(col_nodes)}
    nmax = max(max(row_nodes), max(col_nodes))
    dist = [[0.0 for _ in range(nmax + 1)] for _ in range(nmax + 1)]
    for i in row_nodes:
        for j in col_nodes:
            dist[i][j] = float(mat[ridx[i], cidx[j]])
    return w, v, tw, dist

def parse_route_nodes(route_str: str) -> List[int]:
    nodes = [int(x) for x in str(route_str).split("->")]
    return [x for x in nodes if x != 0]

def read_solution_routes() -> List[Dict[str, object]]:
    if not ROUTE_FILE.exists():
        raise FileNotFoundError(f"未找到路线文件: {ROUTE_FILE}")
    df = pd.read_excel(ROUTE_FILE)
    records: List[Dict[str, object]] = []
    for _, row in df.iterrows():
        nodes = parse_route_nodes(row["virtual_nodes"])
        if not nodes:
            continue
        dep = float(row["departure_hour"]) if "departure_hour" in df.columns else 8.0

```

```

        records.append({"vehicle": str(row["vehicle"]), "route": nodes, "departure_hour": dep})
    return records
def evaluate_plan(
    routes: List[Dict[str, object]],
    w: Dict[int, float],
    v: Dict[int, float],
    tw: Dict[int, Tuple[float, float]],
    dist: List[List[float]],
    fuel_price_factor: float = 1.0,
    elec_price_factor: float = 1.0,
    carbon_price_factor: float = 1.0,
    early_cost_factor: float = 1.0,
    late_cost_factor: float = 1.0,
) -> Dict[str, float]:
    total = {"startup": 0.0, "energy": 0.0, "carbon": 0.0, "tw_cost": 0.0, "distance": 0.0, "total_cost": 0.0}
    for rec in routes:
        veh = VEH_BY_NAME[str(rec["vehicle"])]
        route = list(rec["route"])
        dep = float(rec["departure_hour"])
        ev = eval_route_with_params(
            route,
            veh,
            dep,
            w,
            v,
            tw,
            dist,
            fuel_price_factor=fuel_price_factor,
            elec_price_factor=elec_price_factor,
            carbon_price_factor=carbon_price_factor,
            early_cost_factor=early_cost_factor,
            late_cost_factor=late_cost_factor,
        )
        if ev["feasible"] < 0.5:
            raise RuntimeError("存在路线在参数扰动下不可行，请检查数据与方案。")
        for k in total:
            total[k] += ev[k]
    return total
def build_tw_by_ratio(
    tw: Dict[int, Tuple[float, float]],
    ratio: float,
) -> Dict[int, Tuple[float, float]]:
    new_tw: Dict[int, Tuple[float, float]] = {}
    for c, (a, b) in tw.items():
        center = 0.5 * (a + b)
        half = 0.5 * (b - a) * ratio
        na = max(0.0, center - half)
        nb = min(24.0, center + half)
        if nb <= na:
            nb = na + 1e-3
        new_tw[c] = (na, nb)
    return new_tw
def plot_param_sensitivity(df: pd.DataFrame, out_png: Path) -> None:
    plt.rcParams["font.sans-serif"] = ["Microsoft YaHei", "SimHei", "Noto Sans CJK SC", "Arial Unicode MS"]
    plt.rcParams["axes.unicode_minus"] = False
    fig, ax = plt.subplots(figsize=(10, 6), constrained_layout=True)
    params = ["油价系数", "电价系数", "碳价系数", "早到等待成本系数", "晚到惩罚成本系数"]
    colors = ["#4C78A8", "#F58518", "#54A24B", "#E45756", "#B279A2"]
    markers = ["o", "s", "^", "D", "P"]
    for p, c, m in zip(params, colors, markers):
        sub = df[df["参数"] == p].sort_values("系数")
        ax.plot(
            sub["系数"],
            sub["相对基准变化(%)"],
            marker=m,
            linewidth=2.2,
            markersize=6,
            color=c,
            label=p,
        )
    ax.set_title("成本参数灵敏度分析（单图对比）", fontsize=14)
    ax.set_xlabel("参数系数")
    ax.set_ylabel("总成本相对变化(%)")
    ax.grid(alpha=0.25, linestyle="--")
    ax.axhline(0.0, color="#666666", linewidth=1.0, alpha=0.8)
    ax.legend(title="参数项", loc="best", frameon=True)
    fig.savefig(out_png, dpi=180)
    plt.close(fig)
def plot_tw_sensitivity(df: pd.DataFrame, out_png: Path) -> None:
    plt.rcParams["font.sans-serif"] = ["Microsoft YaHei", "SimHei", "Noto Sans CJK SC", "Arial Unicode MS"]
    plt.rcParams["axes.unicode_minus"] = False
    fig, ax1 = plt.subplots(figsize=(9, 6), constrained_layout=True)
    ax1.plot(
        df["时间窗宽度系数"],
        df["总成本"],
    )

```

```

        marker="o",
        linewidth=3.0,
        markersize=8,
        markerfacecolor="white",
        markeredgewidth=2.0,
        color="#1F4E79",
        label="总成本",
        zorder=3,
    )
    ax1.set_xlabel("时间窗宽度系数 (<1 收紧, >1 放宽)")
    ax1.set_ylabel("总成本 (元)")
    ax1.grid(alpha=0.22, linestyle="--", zorder=0)
    ax1.legend(loc="best", frameon=True)
    ax1.set_title("时间窗宽度灵敏度分析 (总成本)")
    fig.savefig(out_png, dpi=180)
    plt.close(fig)
def main() -> None:
    OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
    w, v, tw, dist = read_inputs()
    routes = read_solution_routes()
    baseline = evaluate_plan(routes, w, v, tw, dist)
    factors = [0.8, 0.9, 1.0, 1.1, 1.2]
    param_specs = [
        ("油价系数", "fuel"),
        ("电价系数", "elec"),
        ("碳价系数", "carbon"),
        ("早到等待成本系数", "early"),
        ("晚到惩罚成本系数", "late"),
    ]
    rows1: List[Dict[str, float]] = []
    for param_name, key in param_specs:
        for fac in factors:
            kw = {
                "fuel_price_factor": 1.0,
                "elec_price_factor": 1.0,
                "carbon_price_factor": 1.0,
                "early_cost_factor": 1.0,
                "late_cost_factor": 1.0,
            }
            if key == "fuel":
                kw["fuel_price_factor"] = fac
            elif key == "elec":
                kw["elec_price_factor"] = fac
            elif key == "carbon":
                kw["carbon_price_factor"] = fac
            elif key == "early":
                kw["early_cost_factor"] = fac
            elif key == "late":
                kw["late_cost_factor"] = fac
            res = evaluate_plan(routes, w, v, tw, dist, **kw)
            rows1.append(
                {
                    "参数": param_name,
                    "系数": fac,
                    "总成本": res["total_cost"],
                    "启动成本": res["startup"],
                    "能耗成本": res["energy"],
                    "碳成本": res["carbon"],
                    "时间窗成本": res["tw_cost"],
                    "相对基准变化(%)": 100.0 * (res["total_cost"] - baseline["total_cost"]) / baseline["total_cost"],
                }
            )
    df_param = pd.DataFrame(rows1)
    df_param.to_excel(OUTPUT_DIR / "q1_灵敏度分析_成本参数.xlsx", index=False)
    plot_param_sensitivity(df_param, OUTPUT_DIR / "q1_灵敏度分析_成本参数.png")
    tw_factors = [0.8, 0.9, 1.0, 1.1, 1.2]
    rows2: List[Dict[str, float]] = []
    for fac in tw_factors:
        tw_new = build_tw_by_ratio(tw, fac)
        res = evaluate_plan(routes, w, v, tw_new, dist)
        rows2.append(
            {
                "时间窗宽度系数": fac,
                "总成本": res["total_cost"],
                "启动成本": res["startup"],
                "能耗成本": res["energy"],
                "碳成本": res["carbon"],
                "时间窗成本": res["tw_cost"],
                "相对基准变化(%)": 100.0 * (res["total_cost"] - baseline["total_cost"]) / baseline["total_cost"],
            }
        )
    df_tw = pd.DataFrame(rows2).sort_values("时间窗宽度系数")
    df_tw.to_excel(OUTPUT_DIR / "q1_灵敏度分析_时间窗宽度.xlsx", index=False)
    plot_tw_sensitivity(df_tw, OUTPUT_DIR / "q1_灵敏度分析_时间窗宽度.png")
    with open(OUTPUT_DIR / "q1_灵敏度分析_结果说明.txt", "w", encoding="utf-8") as f:

```

```
f.write("Q1 灵敏度分析结果说明\n")
f.write(f"基准总成本: {baseline['total_cost']:.2f} 元\n")
f.write("输出文件: \n")
f.write("- q1_灵敏度分析_成本参数.xlsx\n")
f.write("- q1_灵敏度分析_成本参数.png\n")
f.write("- q1_灵敏度分析_时间窗宽度.xlsx\n")
f.write("- q1_灵敏度分析_时间窗宽度.png\n")
print("灵敏度分析完成。")
print(str(OUTPUT_DIR / "q1_灵敏度分析_结果说明.txt"))
if __name__ == "__main__":
    main()
```